



华南理工大学

South China University of Technology

本科毕业设计（论文）

基于编辑流的符号回归

学 院	计算机学院
专 业	计算机科学与技术
学 生 姓 名	王 小 明
学 生 学 号	12350004
指 导 教 师	王大明 (教授)
提 交 日 期	2026 年 3 月 30 日

华南理工大学

学位论文原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果由本人承担。

作者签名：

将这里替换成
您的签名文件



日期：2026 年 3 月 30 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，即：学校有权保存并向国家有关部门或机构送交论文的复印件和电子版，允许学位论文被查阅；学校可以公布学位论文的全部或部分内容，可以允许采用影印、缩印或其它复制手段保存、汇编学位论文。本人电子文档的内容和纸质论文的内容相一致。

作者签名：

将这里替换成
您的签名文件



日期：2026 年 3 月 30 日

指导教师签名：

日期：2026 年 3 月 30 日

作者联系电话：13612345678

电子邮箱：202000000000@mail.scut.edu

摘要

符号回归旨在从数值观测数据中自动发现解析表达式，在科学发现与工程建模中具有重要应用价值。近年来，基于神经网络的端到端符号回归方法因其统一建模和高效推理的优势而受到广泛关注。然而，现有端到端方法主要采用从左到右的序列式生成范式。本文分析发现，这类范式在符号回归中存在两个特有问题：其一，表达式具有前缀树结构，运算符、变量与常数之间存在紧密的层级依赖，早期生成偏差会沿表达式树向下传播，导致结构性误差累积；其二，为弥补上述误差，模型倾向于不断追加冗余子结构来降低残差，生成过长且难以解释的表达式。搜索增强方法虽在一定程度上提升了生成质量，但其搜索过程仍以序列式生成为基础展开策略，未从根本上解决结构修正能力不足的问题。

针对上述问题，本文提出将表达式发现过程建模为对候选表达式的多轮迭代编辑，设计了基于编辑流的端到端符号回归算法 **EditFlowSR**。该方法通过插入、替换和删除三类 **token** 级编辑操作对候选表达式进行全局位置的并行修正，使模型能够在每一轮迭代中同时调整任意位置的符号。为适配符号回归中表达式具有前缀树结构、数值精度要求高以及运算符与常数之间存在语义约束等特点，本文设计了面向数学表达式的编辑操作建模方案，包括基于辅助对齐空间的训练标签构造方法和基于连续时间马尔可夫链的并行编辑推理策略。

本文在三个基准数据集上对 **EditFlowSR** 进行了系统评估，这三个数据集分别包含 119 个费曼方程、14 个 **ODEStrogatz** 挑战和 57 个没有已知底层函数的黑盒回归任务。实验结果表明，在与 **E2E**、**SymbolicGPT** 和 **NeSymReS** 等直接生成式方法的比较中，**EditFlowSR** 在多数设置下取得了更高的 R^2 ，且生成表达式的复杂度更低；在与 **TPSR** 和 **SR4MDL** 等搜索增强式方法的比较中，**EditFlowSR** 的 R^2 处于接近或更高水平，同时推理时间更短。对编辑过程动态行为的分析揭示了表达式发现的三个典型阶段——运算符出现、结构塑造和常数收敛，表明编辑式生成机制能够从随机 **token** 序列出发，通过局部修正逐步构建具有可解释性的数学表达式。

关键词： 符号回归；编辑流；端到端学习；表达式生成

Abstract

Symbolic regression aims to automatically discover analytical expressions from numerical observation data, with important applications in scientific discovery and engineering modeling. In recent years, neural network-based end-to-end symbolic regression methods have attracted widespread attention due to their unified modeling and efficient inference. However, existing end-to-end methods predominantly adopt the left-to-right sequential generation paradigm. This thesis identifies two specific problems of this paradigm in the context of symbolic regression: first, mathematical expressions possess structural integrity constraints where operators, variables, and constants form tightly coupled hierarchical dependencies, such that errors in early generation steps propagate downward through the expression tree, leading to structural error accumulation; second, to compensate for such errors, models tend to append redundant sub-expressions to reduce residuals, producing overly long and uninterpretable expressions. Search-enhanced methods improve generation quality to some extent, yet their search processes still rely on sequential generation as the underlying expansion strategy, without fundamentally addressing the lack of structural correction capability, while introducing additional inference cost.

To address these issues, this thesis proposes a new technical approach: modeling the expression discovery process as iterative editing over candidate expressions, rather than one-shot sequential generation. Specifically, this thesis designs EditFlowSR, an end-to-end symbolic regression algorithm based on edit flows. Instead of expanding expressions along a fixed direction, the method performs parallel position-wise corrections on the current candidate expression through three types of token-level edit operations—insertion, substitution, and deletion—enabling the model to simultaneously adjust symbols at any position in each iteration. To accommodate the characteristics of symbolic regression, such as the prefix-tree structure of expressions, the requirement for numerical precision, and the semantic constraints between operators and constants, this thesis designs an edit operation modeling scheme tailored for mathematical expressions, including a training label construction method based on auxiliary alignment spaces and a parallel editing inference strategy based on continuous-time Markov chains.

This thesis systematically evaluates EditFlowSR on three benchmark datasets comprising 119 Feynman equations, 14 ODEStrogatz challenges, and 57 black-box regression tasks without known underlying functions. Experimental results show that, relative to direct generation methods such as E2E, SymbolicGPT, and NeSymReS, EditFlowSR typically attains higher R^2 scores

and lower expression complexity in the reported settings. Relative to search-enhanced methods such as TPSR and SR4MDL, EditFlowSR yields comparable or higher R^2 with shorter inference time. Furthermore, analysis of the editing dynamics reveals three typical phases of expression discovery—operator emergence, structure shaping, and constant convergence—demonstrating that the edit-based generation mechanism can progressively construct interpretable mathematical expressions from random token sequences through local corrections.

Keywords: Symbolic Regression; Edit Flows; End-to-End Learning; Expression Generation

目 录

摘 要	I
Abstract	II
目 录	IV
插图目录	VI
表格目录	VII
第一章 绪论	1
1.1 引言	1
1.2 研究背景	2
1.3 研究现状	3
1.4 本文工作与论文结构	3
第二章 端到端符号回归相关工作	5
2.1 端到端符号回归的统一架构	5
2.2 表达式生成的训练范式与推理机制	7
2.3 评价指标	10
2.4 本章小结	11
第三章 基于编辑流的端到端符号回归方法	12
3.1 引言	12
3.2 EditFlowSR 模型设计	12
3.3 训练数据构造与模型训练	15
3.4 推理过程	19
3.5 本章小结	21
第四章 实验与结果分析	22
4.1 实验设置	22
4.2 实验结果分析	25
4.3 本章小结	32

结论	33
参考文献	35
致谢	36
附录 A 补充更多细节	37
A.1 补充图	37

插图目录

4-1	EditFlowSR 与直接生成式方法在三类数据集上的性能对比（折线为中位数，阴影区域为 20 次重复实验的 $[0.25, 0.75]$ 分位数区间）	26
4-2	EditFlowSR 与搜索增强式方法在三类数据集上的性能对比（折线为中位数，阴影区域为 20 次重复实验的 $[0.25, 0.75]$ 分位数区间）	28
4-3	噪声强度为 0.1 时六种方法在全部实验组上的综合 Pareto 折中分析	29
4-4	三类编辑操作在归一化 100 步编辑过程中的分布	30
A-1	实验训练集	37
A-2	实验测试集	37

表格目录

4-1	实验数据集特征对比	22
4-2	编辑过程统计概览	30
4-3	编辑操作类型分布统计	30
4-4	Leavitt 数据集上 token 序列的编辑演化	31

第一章 绪论

1.1 引言

符号回归 (Symbolic Regression, SR) 是一类旨在从数值观测数据中自动发现解析表达式的机器学习任务。与传统的参数回归方法不同,符号回归不仅需要确定表达式中的参数取值,还需要同时搜索表达式的函数形式,包括运算符的组合方式、变量的参与结构以及常数的嵌入位置等。因此,符号回归被广泛视为一类同时涉及结构搜索与参数优化的组合优化问题。由于其能够从数据中自动提取具有可解释性的数学规律,符号回归在科学发现、工程建模和数据分析等领域具有重要的应用价值。

传统的符号回归方法主要依赖遗传编程或进化策略在表达式空间中进行搜索^[1]。这类方法通常从随机生成的初始表达式种群出发,通过选择、交叉和变异等遗传操作逐步优化候选表达式。虽然遗传编程方法具有较强的搜索灵活性,但也存在局限:搜索过程需要大量的种群评估,计算开销较高;搜索方向缺乏梯度信息的引导,容易陷入局部最优;且最终结果的表达式质量受超参数设置和随机种子的影响较大,难以保证稳定性。

近年来,随着深度学习技术的发展,基于神经网络的端到端符号回归方法引起了广泛关注。这类方法将符号回归统一建模为从数值观测到解析表达式的条件生成问题,利用 **Transformer** 等神经网络架构直接学习输入输出之间的映射关系^[2]。相较于传统方法,端到端方法具有统一建模、表示学习能力强以及推理速度快等优势,成为当前符号回归领域的研究热点。

然而,现有端到端符号回归方法在表达式生成机制上仍存在局限。从表达式的数学结构特性来看,符号表达式具有严格的前缀树结构,其中运算符、变量和常数之间存在紧密的层级依赖关系。在这种结构下,序列式生成方法按固定顺序逐步展开表达式时,早期步骤中任一符号的生成偏差都会沿表达式树向下传播,导致后续子表达式的语义被连带扭曲,形成结构性的误差累积。为了弥补上述偏差,模型倾向于通过不断追加冗余子结构来降低残差,从而生成过长且难以解释的表达式^[2-3]。搜索增强方法虽然通过引入搜索机制在一定程度上提升了生成质量,但其搜索过程仍以序列式生成作为基础展开策略,并未从根本上解决结构修正能力不足的问题,同时也带来了额外的推理开销^[4-5]。上述分析表明,序列式生成范式在数学表达式这一具有整体结构约束的生成任务中存在本质局限——它缺乏对已生成部分进行回溯修正的机制。因此,如何在保持端到端建模效率的同时,赋予模型对已生成表达式进行结构修正的能力,是当前端到端符号回归中亟需解决的关键问题。

针对上述问题,本文提出一种新的解决思路:将表达式发现过程建模为对候选表达

式的多轮迭代编辑，而非固定顺序的单次生成。与序列式生成只能沿单一方向展开不同，编辑式生成允许模型在每一轮迭代中同时对表达式的任意位置执行插入、替换或删除操作，从而具备了全局范围内的结构修正能力。基于这一思路，本文提出基于编辑流的端到端符号回归算法 **EditFlowSR**，通过连续时间马尔可夫链驱动的并行编辑机制实现候选表达式的迭代优化。

1.2 研究背景

符号回归问题的研究可以追溯到上世纪九十年代。**Koza** 将遗传编程引入符号回归，通过进化操作在表达式空间中搜索最优函数形式，开创了该领域的研究方向。此后数十年间，基于遗传编程的符号回归方法不断发展，涌现出多种改进方案，例如采用多目标优化同时平衡拟合精度与表达式复杂度、引入语义感知的交叉操作以及利用正则化策略控制表达式规模等^[1]。

进入深度学习时代后，端到端符号回归成为新的研究趋势。**Kamienny** 等人提出 **E2E**^[2]，首次将符号回归建模为基于 **Transformer** 的条件序列生成任务，通过端到端训练直接从数值观测生成表达式序列。该方法证明了深度学习模型在大规模合成数据上训练后，能够具备较强的表达式生成能力。随后，**Biggio** 等人提出 **NeSymReS**^[6]，采用 **Set Transformer** 编码器处理观测数据并通过束搜索生成表达式骨架；**Valipour** 等人提出 **SymbolicGPT**^[7]，利用 **GPT** 架构进行表达式生成；**Vastl** 等人提出 **SymFormer**^[3]，探索了符号与常数的联合生成。

在提升生成质量方面，**Shojaee** 等人提出 **TPSR**^[4]，在预训练模型的解码阶段引入蒙特卡洛树搜索，通过前瞻式规划优化解码路径；**Yu** 等人提出 **SR4MDL**^[5]，利用神经网络估计候选表达式的最小描述长度并引导搜索过程。这些方法在一定程度上提升了生成结果的质量，但也引入了额外的搜索开销。

在文本生成领域，编辑式生成作为一种新的序列建模范式逐渐受到关注。与传统自回归生成不同，编辑式方法将序列生成建模为对当前候选序列的逐步修改，通过多轮编辑逐步逼近目标结果。**Havasi** 等人提出 **Edit Flows**^[8]，基于连续时间马尔可夫链对编辑过程进行建模，为序列生成提供了一种区别于自回归的新建模范式。这类方法的核心特征在于：生成过程不再受固定展开方向的约束，每一步都可以对已有序列中的任意位置进行修改。这一特性恰好与符号回归中表达式结构需要反复修正的需求相契合——通过多轮编辑，模型可以逐步调整运算符的选择、变量的组合方式和常数的取值，而不受早期决策的束缚。

1.3 研究现状

当前端到端符号回归方法的研究可以概括为两条主要技术路线。

第一条路线是序列式生成。这类方法将表达式展开为 `token` 序列后，采用标准的自回归解码进行生成。其核心思路是在给定数值条件表示下，按从左到右的顺序逐步预测下一个符号，直至生成完整的表达式。**E2E**^[2]、**NeSymReS**^[6]、**SymbolicGPT**^[7] 和 **SymFormer**^[3] 均属于这一路线。序列式生成方法具有结构统一、推理直接的优势，但其固有局限在于生成过程只能沿固定方向展开，无法回溯修正已生成的符号。

第二条路线是搜索增强推理。这类方法在序列生成的基础上引入搜索机制，在更大的候选空间中探索更优的表达式。**TPSR**^[4] 在预训练模型的推理阶段引入蒙特卡洛树搜索，通过前瞻规划优化生成路径；**SR4MDL**^[5] 则利用最小描述长度作为搜索目标，引导候选表达式的扩展和筛选。搜索增强方法能够提升生成质量，但搜索过程带来的额外计算开销限制了其推理效率。

总体来看，现有方法在表达式生成机制上的共同局限在于：无论是直接生成还是搜索增强，其底层仍然依赖从左到右的序列式生成范式，缺乏对已生成结构进行反复修正的能力。本文提出的基于编辑流的方法则为符号回归提供了一条新的技术路线——通过迭代编辑实现表达式的逐步优化，在保持端到端效率的同时赋予了模型结构修正能力。

1.4 本文工作与论文结构

本文的主要工作如下：

(1) 分析了序列式生成范式在符号回归任务中的结构局限性。本文从表达式的数学结构特性出发，指出其前缀树结构中运算符、变量与常数之间的紧密层级依赖使得序列式生成容易产生结构性误差累积与表达式冗余，为引入编辑式生成提供了面向符号回归任务的具体动机。

(2) 提出了基于编辑流的端到端符号回归算法 **EditFlowSR**，并设计了面向数学表达式的编辑建模方案。该方案将表达式发现过程建模为对候选表达式的多轮迭代编辑，设计了基于辅助对齐空间的编辑标签构造方法以适配前缀表达式的变长对齐需求，并采用基于连续时间马尔可夫链的并行编辑推理策略，使模型能够在每轮迭代中同时对表达式的任意位置进行修正。

(3) 在三个基准数据集上对 **EditFlowSR** 进行了系统的实验评估，这三个数据集分别包含 119 个费曼方程、14 个 **ODEStrogatz** 挑战和 57 个没有已知底层函数的黑盒回归任务；本文从拟合精度、表达式复杂度和推理效率三个维度与多种基线方法进行了比较。实验结果表明，在与直接生成式方法的比较中，**EditFlowSR** 在所报告实验中通常取得更

高的 R^2 且表达式复杂度更低；在与搜索增强式方法的比较中，其 R^2 与基线接近或更高，同时推理时间更短，在拟合精度、表达式简洁性与推理效率三者之间展现出较好的平衡。此外，对编辑过程动态行为的分析揭示了表达式发现的三个典型阶段，并对失败案例进行了初步分析。

本文共分为五章，各章内容安排如下：

第一章为绪论，介绍了符号回归的研究背景和意义，梳理了端到端符号回归的发展历程与当前研究现状，指出了现有方法的不足，并概述了本文的研究内容和论文结构。

第二章为端到端符号回归相关工作，介绍了端到端符号回归的统一架构，包括数值编码器和表达式解码器的基本设计，梳理了序列生成式方法与编辑式生成方法在训练目标和推理机制上的差异，并介绍了本文采用的评价指标。

第三章为基于编辑流的端到端符号回归方法，详细介绍了 **EditFlowSR** 的模型设计、训练数据构造与模型训练以及推理过程。

第四章为实验与结果分析，介绍了实验设置、基线方法及其参数配置，并与直接生成式方法和搜索增强式方法进行了系统比较，同时对编辑过程的动态行为进行了分析。

第五章为结论，总结了本文的研究工作和主要成果，并对未来研究方向进行了展望。

第二章 端到端符号回归相关工作

本节介绍端到端符号回归领域中具有代表性的研究工作，分析其在表达式生成任务中的基本思路与改进方向。首先概述端到端符号回归在表达式生成中的统一框架，归纳其在数值编码、表达式表示和条件生成建模上的共性；随后梳理近期围绕解码器展开的改进，分析序列生成、搜索增强推理和编辑式生成等技术路线的特点与优缺点，为后文本文方法的提出奠定基础。

2.1 端到端符号回归的统一架构

端到端符号回归将符号表达式发现过程统一建模为从数值观测到解析表达式的条件生成问题。设观测数据为 $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ ，其中 $\mathbf{x}_i$ 表示输入向量， y_i 表示对应函数值。现有方法通常先通过数值编码器 $E(\cdot)$ 对观测数据进行表示学习，得到潜在表征 $\mathbf{z} = E(D)$ ，再由解码器 $G(\cdot)$ 基于该表征生成目标表达式 $s = G(\mathbf{z})$ ，等价地可表示为条件分布建模问题 $p(s | D) = p(s | \mathbf{z})$ 。在这一统一框架下，不同方法在数值编码和表达式表示上具有较强共性，因此下面将围绕数值编码器、解码器及其解码机制改进展开介绍。

2.1.1 数值编码器

数值编码器负责对观测点集进行表示学习，将原始数值转换为可用于后续表达式生成的条件表示，记为 $\mathbf{z}$ 。 $\mathbf{z}$ 的具体形式取决于编码器的实现方式，既可以是单个潜在向量，也可以是一组上下文文化特征。其过程通常可分为样本级嵌入和点集级关系建模两个阶段^[9]。首先，对每个观测样本进行样本级嵌入：

$$\mathbf{h}_i = \phi(\mathbf{x}_i, y_i), \quad (2-1)$$

其中， $\phi(\cdot)$ 表示样本级嵌入模块，通常由多层感知机或线性映射实现，用于将输入向量及其对应函数值映射到统一特征空间。通过这一过程，单个观测样本中的局部数值信息被编码为向量表示，从而为后续的全局建模提供基础。

在获得各观测样本的嵌入表示后，模型进一步对全部样本表示进行整体建模：

$$\mathbf{z} = A(\{\mathbf{h}_i\}_{i=1}^N), \quad (2-2)$$

其中， $A(\cdot)$ 表示点集级聚合模块，通常采用基于自注意力机制的编码结构，如 Transformer

encoder。该模块以全部样本表示 $\{h_i\}_{i=1}^N$ 为输入，通过建模不同观测点之间的相关性，提取与目标函数有关的整体变化趋势、局部非线性特征以及变量之间的相互关系，并将这些信息编码为统一的潜在表征 z 。

在具体输入表示上，连续数值通常先经过离散化或科学计数式编码，以缓解直接处理浮点数时可能带来的数值范围过大、表示不稳定以及语义难以建模等问题。常见做法是将数值拆分为符号、尾数和指数等部分分别表示，例如数值 3.14×10^{-2} 可进一步表示为符号 $+$ 、尾数 3.14 和指数 -2 三部分，从而在保留数值精度的同时增强模型对数值结构的刻画能力。

另一方面，由于输入数据本质上表现为观测点集，数值编码阶段还需要适应样本顺序不固定和输入规模变化等特点。为减少无关顺序偏置对表示学习的干扰，部分方法在编码阶段会弱化或移除位置编码，以更好满足观测点集的置换不变性需求；同时，点集级聚合模块直接以样本表示集合 $\{h_i\}_{i=1}^N$ 为输入，其中 N 可随观测数据规模动态变化，因此能够在统一框架下处理不同数量的采样点。

2.1.2 表达式解码器

解码器负责根据数值编码器输出的条件表示生成目标表达式。为便于模型处理，目标表达式通常先被表示为离散符号序列，其中包含操作符、变量、常数以及必要的结构符号。具体实现中，表达式树常通过前序遍历等方式展开为 **token** 序列，从而将表达式生成问题转化为条件序列生成问题。例如，对于表达式 $(x_1 + x_2^2)$ ，若采用前序遍历表示，则可将其展开为符号序列 “ $+$ 、 x_1 、 $\wedge$ 、 x_2 、 2 ”。通过这种方式，原本具有树结构的解析表达式可以转化为神经网络易于处理的离散序列。设解码器记为 $G(\cdot)$ ，则其生成过程可统一表示为 $s = G(z)$ ，其中 z 表示数值编码器输出的条件表示， s 表示目标表达式对应的离散表示序列。解码模块通常采用基于 **Transformer** 的生成结构，在条件表示约束下输出表达式 **token**，并最终恢复出完整的解析表达式。端到端符号回归代表性工作通常都采用这一由条件表示到离散表达式序列的基本框架。

序列生成式解码器将表达式生成视为标准的条件序列生成过程。设目标表达式表示为 $s = (s_1, s_2, \dots, s_T)$ ，则其条件概率通常可写为

$$p(s | z) = \prod_{t=1}^T p(s_t | s_{<t}, z), \quad (2-3)$$

其中， s_t 表示第 t 个生成符号。该类方法根据已生成前缀和条件表示逐步预测下一个 **token**，并从左到右依次完成整个表达式的构造。由于结构统一、推理过程直接，序列生

成式解码器构成了当前端到端符号回归中的基础解码方式，E2E 以及后续的 SymFormer 都属于这一类方法^[2-3]。与此同时，部分方法为了增强解码效果，会在序列生成框架中进一步利用符号回归任务中的拟合误差信息，引入蒙特卡洛树搜索进行前瞻规划，从而对候选表达式的后续展开进行评估并优化解码路径，TPSR^[4] 即属于这一思路。

近期，文本生成领域提出了基于编辑操作的解码机制，将序列生成过程表示为对当前候选序列的逐步修改，而非从空序列开始的逐 token 自回归生成。该类方法目前主要应用于文本生成任务，在符号回归中的应用仍较少，但其通过局部编辑逐步修正候选序列的思想，为表达式生成提供了可借鉴的方向。设当前表达式状态为 $s^{(k)}$ ，一次编辑操作记为 e_k ，则其更新过程可写为

$$s^{(k+1)} = e_k(s^{(k)}). \quad (2-4)$$

在这一框架下，解码器输出的不再只是下一个 token，而是插入、删除、替换等编辑操作，从而通过多步修正逐渐逼近目标序列。例如，当前候选表达式为 $(x_1 + x_2)$ 时，可通过一次替换操作将其中的 x_2 修改为 x_2^2 ，从而得到新的表达式 $(x_1 + x_2^2)$ 。代表性工作如 Levenshtein Transformer 将序列生成建模为插入与删除操作的迭代过程，后续工作则在此基础上进一步扩展了编辑式生成机制。与序列生成方式相比，编辑式解码能够在已有序列基础上进行局部调整，因此为符号回归中的表达式生成提供了一种可借鉴的思路。

总体而言，现有端到端符号回归方法大多可以统一到“数值编码—表达式生成”的框架下：模型首先对输入观测数据进行表示学习，得到用于刻画函数关系的条件表征；随后在该条件表征约束下生成符号表达式。E2E^[2]、SymFormer^[3]、TPSR^[4] 等代表性工作均共享这一编码器—解码器主干，只是在表达式解码阶段的建模方式与推理策略上有所不同。基于此，下面分别从数值编码器与表达式解码器两个方面对相关工作进行介绍。

2.2 表达式生成的训练范式与推理机制

在前文介绍数值编码器与解码器基本框架的基础上，本节进一步围绕两类与本文密切相关的表达式生成机制展开：一类是已经应用于端到端符号回归的序列生成式方法，另一类是主要发展于文本生成领域、但对本文方法具有直接启发意义的编辑式生成方法。

2.2.1 序列生成式方法的训练与推理

对于序列生成式方法，训练阶段通常采用教师强制机制，即在每一个时间步使用真实的目标前缀 $s_{<t}$ 作为解码器输入，以监督模型学习下一个符号的条件分布。设目标表达式序列为 $s = (s_1, s_2, \dots, s_T)$ ，则其训练目标一般定义为 **token** 级交叉熵损失：

$$L_{\text{gen}} = - \sum_{t=1}^T \log p(s_t | s_{<t}, z), \quad (2-5)$$

其中， z 表示数值编码器输出的条件表示， s_t 表示第 t 个目标符号。该损失函数在符号级别最小化预测分布与真实表达式序列之间的差异，使模型能够在数值条件约束下逐步学习表达式的生成规律。

在推理阶段，序列生成式方法不再使用真实前缀，而是将当前时刻之前已经生成的预测结果作为后续输入，按自回归方式逐步生成完整表达式^[2]。其推理过程可写为

$$\hat{s}_t = \arg \max p(s_t | \hat{s}_{<t}, z), \quad (2-6)$$

其中， $\hat{s}_{<t}$ 表示当前时刻之前已经生成的表达式前缀。模型从起始符号开始，依次输出操作符、变量、常数及其他结构符号，直至生成结束标记，从而得到完整的表达式序列。

部分工作进一步引入搜索增强推理机制，以在不改变基础训练范式的前提下提升表达式生成质量^[4]。设基础生成模型在给定条件表示 z 时产生候选表达式 s ，则搜索增强推理的目标可写为

$$\hat{s} = \arg \max_{s \in \mathcal{S}} F(s, z), \quad (2-7)$$

其中， $\mathcal{S}$ 表示由基础生成模型诱导出的候选表达式空间， $F(\cdot)$ 表示用于评价候选表达式质量的综合评分函数。与直接生成式推理仅依据局部符号概率不同，搜索增强推理进一步在候选空间中引入全局规划与路径筛选机制，从而提高最终生成结果的整体质量。

在推理过程中，基础生成模型首先根据当前条件表示给出局部扩展分布，再由搜索算法对候选路径进行展开、评估与选择。以搜索树中的某一中间状态 $s^{(k)}$ 为例，模型可根据当前状态生成后续候选扩展，并在搜索过程中不断更新最优路径。其状态转移过程可抽象表示为

$$s^{(k+1)} \in T(s^{(k)}, z), \quad (2-8)$$

其中， $T(\cdot)$ 表示在条件表示约束下由基础生成模型给出的候选扩展集合。通过对不同候选路径进行递归展开，搜索机制能够在更大范围内探索表达式空间，而不仅局限于单一

步骤上的局部最优选择。

TPSR^[4] 以预训练的 Transformer 符号回归模型作为基础生成器，在推理阶段引入蒙特卡洛树搜索，对候选表达式路径进行前瞻式规划。设当前候选表达式为 s ，其评价函数可写为

$$F(s) = R(s) - \lambda C(s), \quad (2-9)$$

其中， $R(s)$ 表示表达式在观测数据上的拟合质量， $C(s)$ 表示表达式复杂度， λ 为复杂度惩罚系数。搜索过程中，模型不仅考虑当前时刻的符号预测概率，还综合考虑候选表达式在后续展开中的整体表现，从而在拟合精度与表达式简洁性之间取得平衡。通过这种方式，TPSR^[4] 将基础生成模型的局部预测能力与搜索算法的全局规划能力结合起来。然而，这类搜索增强方法的局限在于，虽然其在测试阶段显式引入了拟合精度信息，但随着表达式逐步展开，生成过程对搜索的依赖也不断增强。为了在更大的候选空间中维持结果质量，搜索算法需要持续进行路径扩展、回溯与评估，导致推理时间成本上升，从而在一定程度上削弱了端到端符号回归方法依靠深度模型直接输出表达式的效率优势。

无论是直接生成式方法还是搜索增强方法，其推理过程本质上都仍然遵循左到右的序列式生成范式。该范式的根本局限在于，表达式只能按照 token 顺序逐步展开，早期生成结果会对后续结构形成强约束。一旦模型在前期生成了错误或不理想的符号，后续过程往往难以从整体结构层面对其进行纠正，只能通过继续生成更多符号对已有错误进行补偿。这样虽然可能在表面上改善局部拟合效果，但也容易引入不必要的结构成分，造成表达式冗余，并进一步影响表达式的简洁性与可解释性。

2.2.2 编辑式生成的建模思路及其启发

与前文主要讨论的端到端符号回归方法不同，编辑式生成并非当前符号回归领域中已经广泛采用的主流范式，而是近年来在文本生成任务中发展起来的一类序列建模方法。与序列式生成按固定方向逐 token 展开不同，编辑式生成将序列生成建模为对当前候选序列的逐步修改，通过多轮编辑逐步逼近目标结果。设当前序列状态为 $s^{(k)}$ ，一次编辑操作记为 e_k ，则其更新过程可写为

$$s^{(k+1)} = e_k(s^{(k)}). \quad (2-10)$$

编辑式方法的核心思想在于：解码器不再预测下一个 token，而是预测当前序列应执行的编辑操作（如插入、删除、替换等），通过在已有序列基础上进行局部调整，逐步逼

近目标序列。代表性工作如 **Levenshtein Transformer** 将序列生成建模为插入与删除操作的迭代过程，**Havasi** 等人进一步提出 **Edit Flows**^[8]，基于连续时间马尔可夫链对多步编辑过程进行建模范式。相较于序列式生成，编辑式方法在机制上更有利于序列结构的迭代优化与局部修正，因此为符号表达式发现提供了一种新的建模视角。

2.3 评价指标

本文采用决定系数 R^2 、生成时间和表达式复杂度作为评价指标，分别用于衡量方法的拟合精度、计算效率和表达式简洁性^[9]。

(1) 决定系数 R^2

决定系数用于衡量模型输出的表达式对目标数据变化的解释能力，其定义为

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad (2-11)$$

其中， y_i 表示真实值， $\hat{y}_i$ 表示预测值， $\bar{y}$ 表示真实值的均值。 R^2 越大，说明生成表达式对数据的拟合效果越好；当 $R^2 = 1$ 时表示完全拟合，当 $R^2 = 0$ 时表示模型效果与直接使用均值预测相当。

(2) 生成时间

生成时间用于衡量算法完成符号回归任务并输出最终表达式所需的实际运行时间，单位为秒 (s)。该指标反映了方法的计算效率：生成时间越短，说明算法收敛越快、计算代价越低。对于需要反复展开、筛选和回溯候选表达式的搜索增强方法，这一指标尤其重要。

(3) 表达式复杂度

表达式复杂度用于衡量生成结果的简洁性与可解释性。根据文献 [2], 复杂度定义为最终表达式中 **token** 的总数，即

$$\text{Complexity}(f) = \sum_{j=1}^m I(t_j), \quad (2-12)$$

其中， f 表示生成的表达式， $\{t_j\}_{j=1}^m$ 表示表达式中的各个 **token**，包括系数、运算符和变量， $I(\cdot)$ 为计数函数。复杂度越低，说明表达式越简洁，也通常更容易理解，并且不易因结构过长而产生过拟合现象。

2.4 本章小结

本章总结了端到端符号回归的相关工作，介绍了数值编码器与解码器的基本框架，并梳理了序列生成式方法和编辑式生成方法在训练目标与推理机制上的主要差异。上述分析表明，现有方法在表达式生成机制上存在共同的结构性局限，而编辑式生成在机制上具备对已生成结构进行迭代修正的能力，为后续方法设计提供了基础。

第三章 基于编辑流的端到端符号回归方法

本章针对现有序列生成式符号回归方法中误差累积和结构修正能力不足的问题，提出基于编辑流的端到端符号回归算法 EditFlowSR。

3.1 引言

相较于依赖启发式搜索或进化策略的传统方法，端到端符号回归能够直接学习从数值观测到解析表达式的映射关系。现有代表性工作主要包括序列生成式的 E2E^[2]、SymFormer^[3]，以及在此基础上引入搜索机制的 TPSR^[4]，但总体上仍以序列生成为核心。

然而，符号表达式具有严格的前缀树结构，序列式生成中的早期偏差容易沿层级依赖传播，导致结构性误差累积和表达式冗余；搜索增强方法虽然能提升结果质量，但仍未从根本上解决已有结构难以回溯修正的问题。基于这一分析，本文将表达式发现过程建模为候选表达式的多轮迭代编辑，使模型能够在生成过程中对任意位置进行插入、替换和删除。

为此，本文引入编辑式生成思想，借鉴 Edit Flows^[8] 的编辑流框架，将表达式发现建模为候选表达式的多轮迭代编辑，并提出基于编辑流的端到端符号回归算法 EditFlowSR。本文在已有数值编码方法和编辑流生成框架基础上，构建了适用于前缀表达式的编辑建模方案，包括条件化编辑流解码、辅助对齐空间标签构造和基于连续时间马尔可夫链的并行编辑推理。后续将依次介绍模型设计、训练数据构造与模型训练，以及推理过程。

3.2 EditFlowSR 模型设计

本节介绍本文提出的基于编辑流的端到端符号回归模型 EditFlowSR。从整体上看，EditFlowSR 由数值观测编码模块和编辑流表达式生成模块两部分构成。前者负责对输入观测点集进行表示学习，提取与目标函数关系相关的条件信息；后者则在该条件信息约束下，对当前候选表达式进行逐步更新，从而实现表达式的迭代生成。其中，数值编码模块根据文献 [2] 构建，编辑流解码与连续时间推理部分根据文献 [8] 构建，并结合符号回归任务进行了适配。

3.2.1 数值条件编码

本文采用基于 Transformer 的数值编码器对输入观测数据进行条件表示学习，其设计参考了 E2E^[2] 中的编码方案。该方案已在端到端符号回归中被充分验证，能够有效提取观测数据中蕴含的函数关系特征；同时，本文的研究重点在于表达式生成机制的改进，因此数值编码部分沿用已验证的成熟方案，以便更清晰地隔离和评估编辑式生成机制本身的效果。根据文献^[2]，设输入观测数据经十进制编码后得到数据 token 序列

$$\mathbf{D} = (d_1, d_2, \dots, d_{L_D}), \quad d_j \in \mathcal{V}_{\text{data}}, \quad (3-1)$$

其中， L_D 表示数据序列长度， $\mathcal{V}_{\text{data}}$ 表示数据词表。首先，将离散数据 token 通过嵌入层映射为连续表示，并加入位置信息： $\mathbf{x}_j = \text{Emb}_{\text{data}}(d_j) + \mathbf{p}_j^{\text{data}}$ ，其中 $\mathbf{p}_j^{\text{data}} \in \mathbb{R}^{d_e}$ 表示位置 j 的位置编码。随后，将输入矩阵 $\mathbf{X} \in \mathbb{R}^{L_D \times d_e}$ 输入 Transformer 编码器，得到上下文化隐藏表示 $\mathbf{H} = \text{Encoder}(\mathbf{X}) \in \mathbb{R}^{L_D \times d_e}$ ，并通过注意力读出机制聚合为全局条件表示：

$$\alpha_j = \frac{\exp(\mathbf{w}_a^\top \mathbf{h}_j)}{\sum_{k=1}^{L_D} \exp(\mathbf{w}_a^\top \mathbf{h}_k)}, \quad \mathbf{g} = \sum_{j=1}^{L_D} \alpha_j \mathbf{h}_j, \quad \mathbf{z} = \mathbf{W}_b \mathbf{g} + \mathbf{b}_b \in \mathbb{R}^{d_z}. \quad (3-2)$$

其中， $\mathbf{w}_a \in \mathbb{R}^{d_e}$ 为注意力读出参数， $\mathbf{z} \in \mathbb{R}^{d_z}$ 为最终的数值条件向量， $\mathbf{W}_b \in \mathbb{R}^{d_z \times d_e}$ 和 $\mathbf{b}_b \in \mathbb{R}^{d_z}$ 分别表示线性映射的权重矩阵和偏置向量。数值编码器将输入观测数据映射为条件向量 $\mathbf{z}$ ，并将其作为后续编辑流表达式生成模块的条件输入。

3.2.2 条件化的编辑流解码器

为便于模型处理，本文将表达式树通过前序遍历展开为表达式 token 序列。设第 r 轮编辑迭代时的当前候选表达式为

$$\mathbf{s}^{(r)} = (s_1^{(r)}, s_2^{(r)}, \dots, s_{L_r}^{(r)}), \quad s_j^{(r)} \in \mathcal{V}_{\text{exp}}, \quad (3-3)$$

其中， L_r 表示第 r 轮候选表达式的序列长度， $\mathcal{V}_{\text{exp}}$ 表示表达式词表。编辑流解码器的目标是在数值条件向量 $\mathbf{z}$ 的约束下，对当前候选表达式各位置的编辑行为进行建模，而不是按自回归方式直接预测下一个 token。

首先，对当前候选表达式进行嵌入表示。设解码器隐藏维度为 d_h ，则第 j 个位置的

输入表示及其序列矩阵可写为

$$\mathbf{e}_j^{(r)} = \text{Emb}_{\text{exp}}(s_j^{(r)}) + \mathbf{p}_j^{\text{exp}}, \quad \mathbf{E}^{(r)} = \begin{bmatrix} (\mathbf{e}_1^{(r)})^\top \\ \vdots \\ (\mathbf{e}_{L_r}^{(r)})^\top \end{bmatrix} \in \mathbb{R}^{L_r \times d_h}. \quad (3-4)$$

其中, $\text{Emb}_{\text{exp}}(\cdot)$ 表示表达式 token 的嵌入层, 用于将离散符号映射为 d_h 维连续向量; $\mathbf{p}_j^{\text{exp}} \in \mathbb{R}^{d_h}$ 表示位置 j 的位置编码; $\mathbf{e}_j^{(r)} \in \mathbb{R}^{d_h}$ 表示第 r 轮编辑中位置 j 的输入表示。

随后, 将数值条件映射到解码器隐空间, 并与表达式表示一起输入条件化解码器:

$$\mathbf{c} = \mathbf{W}_z \mathbf{z} + \mathbf{b}_z \in \mathbb{R}^{d_h}, \quad \mathbf{U}^{(r)} = \text{Decoder}(\mathbf{E}^{(r)}, \mathbf{c}) = \begin{bmatrix} (\mathbf{u}_1^{(r)})^\top \\ \vdots \\ (\mathbf{u}_{L_r}^{(r)})^\top \end{bmatrix} \in \mathbb{R}^{L_r \times d_h}. \quad (3-5)$$

其中, $\mathbf{c} \in \mathbb{R}^{d_h}$ 表示投影后的条件表示; $\mathbf{W}_z \in \mathbb{R}^{d_h \times d_z}$ 和 $\mathbf{b}_z \in \mathbb{R}^{d_h}$ 分别表示条件投影的权重矩阵和偏置向量; $\mathbf{u}_j^{(r)} \in \mathbb{R}^{d_h}$ 表示解码器输出的第 j 个位置隐藏状态。该解码器各层由自注意力、条件注意力和前馈网络构成, 其中自注意力用于建模当前候选表达式内部的依赖关系, 条件注意力用于在数值条件 $\mathbf{c}$ 的约束下生成编辑相关的上下文特征。

根据文献 [8], 在编辑流解码过程中还需要显式引入时间步信息。设 $t \in [0, 1]$ 表示编辑流对应的时间步, 则其时间嵌入定义为

$$\boldsymbol{\tau}(t) = \text{Emb}_t(t) \in \mathbb{R}^{d_h}. \quad (3-6)$$

其中, $\text{Emb}_t(\cdot)$ 表示时间步嵌入函数。时间步 t 先经过正弦时间编码, 再通过带 SiLU 激活函数的两层感知机映射到解码器隐空间。这里不再展开具体的时间调度公式, 仅保留其抽象表示。

最后, 模型基于解码器输出和时间嵌入, 对每个位置的编辑操作进行预测。记

$$\boldsymbol{\eta}_j^{(r)} = [\mathbf{u}_j^{(r)}; \boldsymbol{\tau}(t)] \in \mathbb{R}^{2d_h}, \quad (3-7)$$

则第 j 个位置的编辑速率和 token 分布预测为

$$\boldsymbol{\lambda}_j^{(r)} = \text{softplus}(\text{Head}_{\text{rate}}(\boldsymbol{\eta}_j^{(r)})) \in \mathbb{R}^3, \quad (3-8)$$

$$\mathbf{p}_{j,\text{ins}}^{(r)} = \text{softmax}\left(\text{Head}_{\text{ins}}(\boldsymbol{\eta}_j^{(r)})\right) \in \mathbb{R}^{|\mathcal{V}_{\text{exp}}|}, \quad \mathbf{p}_{j,\text{sub}}^{(r)} = \text{softmax}\left(\text{Head}_{\text{sub}}(\boldsymbol{\eta}_j^{(r)})\right) \in \mathbb{R}^{|\mathcal{V}_{\text{exp}}|}. \quad (3-9)$$

其中, $[\cdot; \cdot]$ 表示向量拼接操作; $\boldsymbol{\lambda}_j^{(r)} = (\lambda_{j,\text{ins}}^{(r)}, \lambda_{j,\text{sub}}^{(r)}, \lambda_{j,\text{del}}^{(r)})$ 分别表示第 j 个位置的插入速率、替换速率和删除速率; $\mathbf{p}_{j,\text{ins}}^{(r)}$ 和 $\mathbf{p}_{j,\text{sub}}^{(r)}$ 分别表示插入 token 与替换 token 的条件分布。删除操作只需预测其发生强度, 因此不再额外预测目标 token。三个预测头均采用两层前馈网络实现, 中间使用 SiLU 激活函数。

3.3 训练数据构造与模型训练

3.3.1 训练数据生成

本文训练数据的生成过程遵循文献 [2]。对于每个训练样本, 首先随机生成一个目标函数 $f: \mathbb{R}^D \rightarrow \mathbb{R}$, 再围绕该函数采样输入点并计算对应输出, 最终构造监督样本

$$(D_f, s^*), \quad D_f = \{(x_i, y_i)\}_{i=1}^N, \quad y_i = f(x_i), \quad (3-10)$$

其中, D_f 表示函数 f 对应的数值观测集合, $x_i \in \mathbb{R}^D$ 和 $y_i \in \mathbb{R}$ 分别表示第 i 个输入点及其函数值, s^* 表示目标表达式对应的 token 序列。

目标函数通过随机表达式树生成。具体而言, 首先随机确定输入维度以及表达式中一元、二元运算符的数量, 再以数学运算符作为内部节点、变量或常数作为叶节点构造表达式树, 并在树的随机位置插入一元运算符。为增强表达式在数值尺度上的多样性, 还对变量项及部分子表达式施加随机仿射变换。通过上述过程, 可以在控制表达式复杂度的同时生成大量结构多样、参数多样的目标函数, 从而降低模型对有限模板的记忆风险。

在目标函数确定后, 进一步为其生成数值观测数据。对于每个函数, 随机采样一定数量的输入点, 并计算对应的函数值。输入点并非来自单一分布, 而是从由多个簇构成的混合分布中采样, 各簇的中心、尺度以及分布形状均随机生成, 分布类型可为高斯分布或均匀分布。随后, 对采样点施加随机旋转, 并按维度进行中心化与标准化处理, 以提高训练数据在空间分布、尺度范围和局部密度上的多样性。若采样点超出目标函数的定义域, 或对应函数值发生数值溢出, 则舍弃当前样本并重新生成目标函数。

在得到数值观测与目标函数后, 还需将其转换为模型可处理的序列形式。数值采用十进制浮点表示编码为 token 序列, 表达式树则按前序遍历展开为前缀表达式序列。例如, 表达式 $f(x) = \cos(2.4242x)$ 可表示为 $[\cos, \text{mul}, +, 2424, \text{E}-3, x]$ 。由此, 原始函数生成过程被统一转换为“数值观测–目标表达式”的监督样本对, 为后续模型训练提供数

据基础。

3.3.2 辅助对齐空间与训练标签的构造

由第 3.2 节可知, EditFlowSR 在每个位置输出插入、替换和删除三类编辑操作的发生速率, 以及插入 token 与替换 token 的条件分布。因此, 在训练阶段需要构造的监督信号不再是传统自回归生成中的下一个 token, 而是给定当前候选表达式时, 各位置应执行何种编辑操作, 以及对应的目标 token 是什么。由于模型实际接收的输入是当前候选表达式, 因此训练标签也需要由当前表达式与目标表达式之间的差异来构造。根据文献 [8], 这里引入辅助对齐空间, 并针对前缀表达式的变长对齐进行了改写。

首先, 定义表达式序列空间。设表达式 token 词表为 $\mathcal{V}_{\text{exp}}$, 允许的最大表达式长度为 N , 则表达式序列空间可表示为

$$\mathcal{X} = \bigcup_{n=0}^N \mathcal{V}_{\text{exp}}^n. \quad (3-11)$$

设第 r 轮编辑迭代时输入模型的当前候选表达式为 $\mathbf{s}^{(r)} = (s_1^{(r)}, s_2^{(r)}, \dots, s_{n_r}^{(r)}) \in \mathcal{X}$, 目标表达式为 $\mathbf{s}^* = (s_1^*, s_2^*, \dots, s_{n^*}^*) \in \mathcal{X}$, 其中 $\mathbf{s}^{(r)}$ 表示第 r 轮编辑状态下的当前表达式, $\mathbf{s}^*$ 表示与观测数据 D_f 对应的真实目标表达式。

为便于在变长序列之间构造位置级监督信号, 进一步在词表 $\mathcal{V}_{\text{exp}}$ 之外引入空白符 ε , 并考虑辅助对齐空间

$$\mathcal{Z} = (\mathcal{V}_{\text{exp}} \cup \{\varepsilon\})^L, \quad (3-12)$$

其中, L 表示对齐后的公共长度, ε 仅用于序列对齐与标签构造, 不属于真实表达式词表, 也不会作为模型的输入或输出。

给定当前表达式 $\mathbf{s}^{(r)}$ 和目标表达式 $\mathbf{s}^*$ 后, 首先利用 Levenshtein 编辑距离对应的动态规划对齐算法, 构造一组等长对齐序列

$$\mathbf{z}^{(r)} = (z_1^{(r)}, z_2^{(r)}, \dots, z_L^{(r)}), \quad \mathbf{z}^* = (z_1^*, z_2^*, \dots, z_L^*), \quad (3-13)$$

其中, $\mathbf{z}^{(r)}, \mathbf{z}^* \in \mathcal{Z}$, 并满足从二者中删去全部空白符 ε 后, 分别恢复为原始序列 $\mathbf{s}^{(r)}$ 和 $\mathbf{s}^*$ 。经过这一步, 原本定义在两个变长表达式之间的编辑关系就被转化为对齐位置上的符号对应关系。若存在多条最优对齐路径, 则在实现中固定采用同一回溯规则, 以保证相同样本始终生成一致的训练标签。

由于模型的预测是基于当前表达式 $\mathbf{s}^{(r)}$ 的位置进行的, 因此还需将辅助对齐空间中

的位置映射回当前表达式中的位置索引。为此，定义

$$\pi(i) = \sum_{k=1}^i \mathbf{1}[z_k^{(r)} \neq \varepsilon], \quad \bar{\pi}(i) = \sum_{k=1}^{i-1} \mathbf{1}[z_k^{(r)} \neq \varepsilon]. \quad (3-14)$$

其中， $\pi(i)$ 表示对齐位置 i 在当前表达式 $\mathbf{s}^{(r)}$ 中对应的 token 位置； $\bar{\pi}(i)$ 表示该对齐位置左侧已经出现的可见 token 数量，因此可作为插入操作的锚点位置。当 $\bar{\pi}(i) = 0$ 时，表示在序列开头执行插入。

在此基础上，便可根据对齐结果分别构造插入、替换和删除三类监督标签。

(a) 插入标签

若某个对齐位置满足

$$z_i^{(r)} = \varepsilon, \quad z_i^* \neq \varepsilon, \quad (3-15)$$

则说明目标表达式在该位置相较于当前表达式多出一个 token，此时应构造插入标签，表示在位置 $\bar{\pi}(i)$ 之后插入 token z_i^* 。对应的插入标签集合为

$$Y_{\text{ins}} = \left\{ (\bar{\pi}(i), z_i^*) \mid z_i^{(r)} = \varepsilon, z_i^* \neq \varepsilon \right\}. \quad (3-16)$$

(b) 替换标签

若某个对齐位置满足

$$z_i^{(r)} \neq \varepsilon, \quad z_i^* \neq \varepsilon, \quad z_i^{(r)} \neq z_i^*, \quad (3-17)$$

则说明当前表达式与目标表达式在该位置的 token 不一致，此时应构造替换标签，表示将当前表达式第 $\pi(i)$ 个 token 替换为 z_i^* 。对应的替换标签集合为

$$Y_{\text{sub}} = \left\{ (\pi(i), z_i^*) \mid z_i^{(r)} \neq \varepsilon, z_i^* \neq \varepsilon, z_i^{(r)} \neq z_i^* \right\}. \quad (3-18)$$

(c) 删除标签

若某个对齐位置满足

$$z_i^{(r)} \neq \varepsilon, \quad z_i^* = \varepsilon, \quad (3-19)$$

则说明当前表达式在该位置存在多余 token，此时应构造删除标签，表示删除当前表达式第 $\pi(i)$ 个 token。对应的删除标签集合为

$$Y_{\text{del}} = \left\{ \pi(i) \mid z_i^{(r)} \neq \varepsilon, z_i^* = \varepsilon \right\}. \quad (3-20)$$

若某个对齐位置满足

$$z_i^{(r)} = z_i^*, \quad (3-21)$$

则说明当前表达式与目标表达式在该位置已经一致，此时无需为该位置构造正样本编辑标签。

3.3.3 损失函数

在完成第 3.3.2 节中的辅助对齐与训练标签构造后，模型训练目标可表述为基于条件编辑率的损失函数。根据文献 [8]，对于当前表达式 $\mathbf{s}^{(r)}$ 及其与目标表达式 $\mathbf{s}^*$ 的对齐结果 $\mathbf{z}^{(r)}$ 和 $\mathbf{z}^*$ ，损失函数定义为

$$\mathcal{L}(\theta) = \mathbb{E} \left[\sum_{\mathbf{s} \neq \mathbf{s}^{(r)}} u_{\theta}(\mathbf{s} | \mathbf{s}^{(r)}) - \sum_{i=1}^L \mathbf{1}[z_i^{(r)} \neq z_i^*] \omega_r \log u_{\theta}(\mathbf{s}(\mathbf{z}^{(r)}, i, z_i^*) | \mathbf{s}^{(r)}) \right]. \quad (3-22)$$

其中， $u_{\theta}(\mathbf{s} | \mathbf{s}^{(r)})$ 表示模型从当前表达式 $\mathbf{s}^{(r)}$ 转移到表达式 $\mathbf{s}$ 的条件编辑率， L 表示对齐序列长度， $\mathbf{1}[\cdot]$ 为示性函数， ω_r 为由训练调度确定的权重系数； $\mathbf{s}(\mathbf{z}^{(r)}, i, z_i^*)$ 表示将对齐序列 $\mathbf{z}^{(r)}$ 的第 i 个位置修改为目标符号 z_i^* 后，再删去全部空白符 ε 所得到的表达式状态，即从当前表达式出发沿目标方向执行一次正确编辑后的后继状态。

式中第一项对当前表达式下所有可能的后继编辑率求和，用于抑制模型产生过多无效编辑；第二项仅对当前仍未与目标表达式一致的位置进行监督，用于提高真实编辑路径上正确操作的转移率。因此，该损失函数同时兼顾了压低无效编辑和强化正确编辑两个目标。

结合第 3.2 节中的模型输出形式，转移率 $u_{\theta}(\mathbf{s} | \mathbf{s}^{(r)})$ 可由三类局部编辑操作参数化

$$u_{\theta}(\text{ins}(\mathbf{s}^{(r)}, i, a) | \mathbf{s}^{(r)}) = \lambda_{i,\text{ins}}^{(r)} \mathcal{Q}_{i,\text{ins}}^{(r)}(a), \quad u_{\theta}(\text{sub}(\mathbf{s}^{(r)}, i, a) | \mathbf{s}^{(r)}) = \lambda_{i,\text{sub}}^{(r)} \mathcal{Q}_{i,\text{sub}}^{(r)}(a), \quad (3-23)$$

$$u_{\theta}(\text{del}(\mathbf{s}^{(r)}, i) | \mathbf{s}^{(r)}) = \lambda_{i,\text{del}}^{(r)}. \quad (3-24)$$

其中， $\lambda_{i,\text{ins}}^{(r)}$ 、 $\lambda_{i,\text{sub}}^{(r)}$ 和 $\lambda_{i,\text{del}}^{(r)}$ 分别表示当前位置执行插入、替换和删除操作的速率， $\mathcal{Q}_{i,\text{ins}}^{(r)}(\cdot)$ 与 $\mathcal{Q}_{i,\text{sub}}^{(r)}(\cdot)$ 分别表示插入 token 和替换 token 的条件分布。

由于 $\mathcal{Q}_{i,\text{ins}}^{(r)}(\cdot)$ 和 $\mathcal{Q}_{i,\text{sub}}^{(r)}(\cdot)$ 均为归一化概率分布，因此总转移率项可进一步化为

$$\sum_{\mathbf{s} \neq \mathbf{s}^{(r)}} u_{\theta}(\mathbf{s} | \mathbf{s}^{(r)}) = \sum_{j=0}^{n_r} \lambda_{j,\text{ins}}^{(r)} + \sum_{j=1}^{n_r} \lambda_{j,\text{sub}}^{(r)} + \sum_{j=1}^{n_r} \lambda_{j,\text{del}}^{(r)}, \quad (3-25)$$

其中, n_r 表示当前表达式 $\mathbf{s}^{(r)}$ 的长度。由此可见, 原始定义在状态空间上的损失函数最终可以转化为对各位置局部编辑速率的求和, 从而避免显式枚举全部后继表达式。

进一步结合第 3.3.2 节中的标签构造方式可知, 式中的正确转移项在实现时分别对应插入、替换和删除三类局部编辑监督。其中, 插入和替换标签同时监督编辑位置的操作强度与目标 token 分布, 删除标签仅监督对应位置的删除速率。训练时, 对一个 batch 内所有样本的损失取平均, 并采用反向传播更新模型参数, 从而学习在给定数值观测条件下, 从当前表达式 $\mathbf{s}^{(r)}$ 出发逐步逼近目标表达式 $\mathbf{s}^*$ 的条件化编辑策略。

3.4 推理过程

在完成模型训练后, EditFlowSR 的推理目标是在给定数值观测 D_f 的条件下, 从初始候选表达式出发, 通过多轮局部编辑逐步得到最终的目标表达式。与传统自回归方法按固定顺序逐 token 生成不同, 根据文献 [8], 推理过程建模为定义在表达式状态空间 $\mathcal{X}$ 上的连续时间随机编辑过程: 模型在当前表达式状态下预测各位置执行插入、替换和删除操作的发生强度, 并据此对表达式进行迭代更新。

首先, 给定输入观测数据 D_f , 利用第 3.2.1 节中的数值编码器得到条件向量 $\mathbf{z}$ 。随后选定初始候选表达式 $\mathbf{s}^{(0)}$ 。本文中, $\mathbf{s}^{(0)}$ 可以取为空序列, 也可以取为某个简单模板表达式。设第 r 轮编辑时的当前表达式为

$$\mathbf{s}^{(r)} = (s_1^{(r)}, s_2^{(r)}, \dots, s_{n_r}^{(r)}), \quad (3-26)$$

其中 n_r 表示当前表达式长度。将 $\mathbf{s}^{(r)}$ 、条件向量 $\mathbf{z}$ 以及对应时间步 t_r 输入第 3.2.2 节的条件化编辑流解码器后, 模型会输出当前位置的三类编辑速率

$$\lambda_{j,\text{ins}}^{(r)}, \quad \lambda_{j,\text{sub}}^{(r)}, \quad \lambda_{j,\text{del}}^{(r)}, \quad (3-27)$$

以及插入、替换操作对应的 token 条件分布

$$p_{j,\text{ins}}^{(r)}(\cdot), \quad p_{j,\text{sub}}^{(r)}(\cdot). \quad (3-28)$$

由此，当前状态下的条件编辑率可由局部编辑操作参数化表示为

$$u_{\theta}(\text{ins}(\mathbf{s}^{(r)}, j, a) | \mathbf{s}^{(r)}) = \lambda_{j,\text{ins}}^{(r)} p_{j,\text{ins}}^{(r)}(a), \quad j = 0, 1, \dots, n_r, \quad (3-29)$$

$$u_{\theta}(\text{sub}(\mathbf{s}^{(r)}, j, a) | \mathbf{s}^{(r)}) = \lambda_{j,\text{sub}}^{(r)} p_{j,\text{sub}}^{(r)}(a), \quad j = 1, 2, \dots, n_r, \quad (3-30)$$

$$u_{\theta}(\text{del}(\mathbf{s}^{(r)}, j) | \mathbf{s}^{(r)}) = \lambda_{j,\text{del}}^{(r)}, \quad j = 1, 2, \dots, n_r. \quad (3-31)$$

其中, $\text{ins}(\mathbf{s}^{(r)}, j, a)$ 表示在当前位置锚点 j 之后插入 token a 所得到的新表达式; $\text{sub}(\mathbf{s}^{(r)}, j, a)$ 表示将第 j 个 token 替换为 a ; $\text{del}(\mathbf{s}^{(r)}, j)$ 表示删除第 j 个 token。根据第 3.3.3 节的定义，当前状态下的总编辑强度为

$$\Lambda^{(r)} = \sum_{j=0}^{n_r} \lambda_{j,\text{ins}}^{(r)} + \sum_{j=1}^{n_r} \lambda_{j,\text{sub}}^{(r)} + \sum_{j=1}^{n_r} \lambda_{j,\text{del}}^{(r)}. \quad (3-32)$$

它描述了当前表达式在单位时间内发生任意一次编辑的总体倾向。

从随机过程角度看，式 (3-29) 式 (3-31) 给出的并不是静态分数，而是连续时间马尔可夫链中的转移率。因此，可以将每个位置上的插入、替换和删除操作进一步理解为由强度函数驱动的局部非齐次泊松计数过程。记 $\text{op} \in \{\text{ins}, \text{sub}, \text{del}\}$ ，并令 $N_{j,\text{op}}(t)$ 表示第 j 个位置上对应编辑操作在时间 t 之前的累计发生次数，则在一个很小的时间区间 $(t_r, t_r + h)$ 内，有

$$N_{j,\text{op}}(t_r + h) - N_{j,\text{op}}(t_r) \sim \text{Poisson}\left(\int_{t_r}^{t_r+h} \lambda_{j,\text{op}}(\xi) d\xi\right). \quad (3-33)$$

因此，该时间区间内至少发生一次相应编辑的概率可写为

$$\Pr(N_{j,\text{op}}(t_r + h) - N_{j,\text{op}}(t_r) \geq 1 | \mathbf{s}^{(r)}) = 1 - \exp\left(-\int_{t_r}^{t_r+h} \lambda_{j,\text{op}}(\xi) d\xi\right) \approx h \lambda_{j,\text{op}}^{(r)}. \quad (3-34)$$

当步长 h 足够小时，同一位置在一个微小时间区间内发生两次及以上编辑的概率为高阶小量，可以忽略不计。这样一来，连续时间下的编辑流推理就可以近似转化为离散步长上的伯努利采样过程：在每一轮中，根据 $h\lambda_{j,\text{op}}^{(r)}$ 决定相应编辑是否发生。上述近似正是本文由 CTMC 速率到可实现采样算法的核心桥梁。

在具体实现时，本文采用固定步长的并行编辑近似推理。设总迭代轮数为 R ，则令 $h = 1/R$ ，并令第 r 轮对应时间步为 $t_r = rh$ 。对当前表达式 $\mathbf{s}^{(r)}$ ，首先对每个插入位置 $j = 0, 1, \dots, n_r$ 按概率 $h\lambda_{j,\text{ins}}^{(r)}$ 采样是否执行插入；再对每个已有 token 位置 $j = 1, 2, \dots, n_r$ 按概率 $h(\lambda_{j,\text{sub}}^{(r)} + \lambda_{j,\text{del}}^{(r)})$ 采样该位置是否触发“替换或删除”事件。若事件触发，则进一

步按

$$\frac{\lambda_{j,\text{del}}^{(r)}}{\lambda_{j,\text{sub}}^{(r)} + \lambda_{j,\text{del}}^{(r)}} \quad (3-35)$$

的概率选择删除操作，否则选择替换操作。若某位置被判定执行插入或替换，则分别从分布 $p_{j,\text{ins}}^{(r)}(\cdot)$ 或 $p_{j,\text{sub}}^{(r)}(\cdot)$ 中采样目标 **token**。由此即可得到当前轮的编辑集合 $E^{(r)}$ 。最后，将 $E^{(r)}$ 中的操作同时施加到 $\mathbf{s}^{(r)}$ 上，得到下一轮候选表达式

$$\mathbf{s}^{(r+1)} = \Phi(\mathbf{s}^{(r)}, E^{(r)}), \quad (3-36)$$

其中 $\Phi(\cdot)$ 表示并行编辑更新算子。通过这种“预测局部编辑强度—采样编辑事件—同步更新表达式”的方式，模型能够在已有候选表达式基础上持续进行局部修正，而不是像自回归方法那样只能单向展开。

当满足以下任一条件时，推理过程终止：1) 达到预设最大迭代轮数 R ；2) 表达式已经满足外部定义的结束判据。最终输出 $\mathbf{s}^{(R)}$ 或最后一个非空编辑状态作为预测表达式。

3.5 本章小结

针对现有序列生成式符号回归方法结构修正能力不足的问题，本文提出了基于编辑流的端到端符号回归算法 **EditFlowSR**。本文将表达式发现建模为候选表达式的多轮迭代编辑，并结合已有数值编码方法与编辑流生成框架，构建了适用于前缀表达式的符号回归编辑建模方案，包括条件化编辑流解码、辅助对齐空间标签构造和基于连续时间马尔可夫链的并行编辑推理，从而增强了模型对表达式整体结构的修正能力。

第四章 实验与结果分析

本章通过系统实验验证本文所提 EditFlowSR 的性能，分别从与直接生成式方法的比较、与搜索增强式方法的比较两个方面展开，重点考察本文方法在拟合精度、表达式复杂度、生成时间及噪声条件下的综合表现。

4.1 实验设置

4.1.1 数据集

本文采用符号回归领域广泛使用的三个基准数据集进行实验评估，分别是包含 119 个费曼方程的 Feynman 数据集、包含 14 个 ODEStrogatz 挑战的 Strogatz 数据集，以及包含 57 个没有已知底层函数的黑盒回归任务的 Black-box 数据集。这些数据集涵盖了不同复杂度和领域的回归问题。表 4-1 总结了三个数据集的主要特征。

表 4-1 实验数据集特征对比

数据集	任务数量	数据性质	输入维度	单任务样本规模
Feynman	119	Ground-truth	$D \leq 10$	10^5
Strogatz	14	Ground-truth	$D = 2$	400
Black-box	57	Unknown ground-truth	$D \leq 10$	47–40K

Feynman 数据集¹: Feynman 数据集来源于 Feynman Lectures on Physics ^[10]，共包含 119 个物理方程。这些方程既反映了客观的物理规律，且在结构形式上具有较强多样性，因此该数据集常被用作评估符号回归方法发现科学规律能力的重要基准。为了保证不同任务之间评测规模的一致性，数据集将输入维度限制为 $D \leq 10$ 。此外，该数据集提供真实的目标函数，能够避免黑盒数据集中常见的评价歧义。整个数据集共包含约 10^5 个数据点，因此也适合用于检验模型在大规模回归任务中的准确性和泛化能力。

Strogatz 数据集²: Strogatz 数据集来源于非线性动力系统问题，共包含 14 个 ODE-Strogatz 挑战，因此输入维度固定为 $D = 2$ 。与更强调物理公式可解释性的 Feynman 数据集不同，Strogatz 数据集更关注动态行为的刻画，尤其适合用于检验模型对非线性时变规律的建模能力。每个任务包含 400 个样本，并提供对应的真实函数，便于在明确的动力系统背景下对模型性能进行评估。因此，该数据集在小样本和强非线性条件下的符号回归性能测试中具有较高代表性。

¹<https://space.mit.edu/home/tegmark/aifeynman.html>

²<https://github.com/lacava/ode-strogatz>

Black-box 数据集¹: Black-box 数据集主要来自 Penn Machine Learning Benchmark (PMLB) [11], 并被 SRBench [1] 纳入评测基准, 用于衡量符号回归方法在复杂回归任务中的表现。与 Feynman 和 Strogatz 这类已知真实表达式的数据集不同, Black-box 数据集不提供真实底层函数, 更接近实际应用场景, 因此更适合用来考察模型的拟合能力、泛化能力和可解释性。该基准共包含 57 个没有已知底层函数的黑盒回归任务, 输入维度限制为 $D \leq 10$ 。该组数据既包含真实世界数据, 也包含合成数据, 任务之间在噪声水平、数据规模和问题难度上差异较大, 因此能够更全面地检验方法的稳健性。其样本规模变化范围也较大, 大致分布在 47 到 40K 之间。

总体来看, Feynman 数据集更侧重物理规律的可解释性, Strogatz 数据集关注非线性动力系统建模能力, 而 Black-box 数据集则更强调真实场景下的泛化性能。在评估过程中, 对于每个任务, 从原始数据集中随机采样 200 个数据点, 按 75%:25% 的比例划分为训练集和测试集, 所有方法报告的 R^2 均在测试集上计算。

4.1.2 实验环境与实现细节

本文实验在一台 Ubuntu 22.04.5 LTS 服务器上进行, 操作系统内核版本为 Linux 6.8.0-48-generic。服务器 CPU 为 Intel Xeon Gold 6226R @ 2.90GHz, 共 32 个逻辑核心; 系统内存为 125 GiB。GPU 配置为 4 张 NVIDIA GeForce RTX 3090 显卡, 每张显存 24 GB; NVIDIA 驱动版本为 560.35.03, 系统 CUDA 版本为 12.6。

软件环境采用 Python 3.11.14 与 PyTorch 2.10.0+cu128, cuDNN 版本为 91002; 在该环境下, PyTorch 可正常识别全部 4 张 GPU。为保证对比实验的一致性, 本文报告的各种方法测试与计时结果均统一在该服务器上完成。

EditFlowSR 的训练同样在该服务器上进行, 实际使用其中 3 张 GPU。训练数据采用在线生成方式, 即训练过程中按需动态生成监督样本而不预先离线存储。训练时全局 batch size 设为 112, 并采用 FP16 混合精度训练。训练过程中每 5 个 epoch 进行一次验证, 并根据验证结果更新最优权重; 在该设置下, 最优模型于约 29.1 h 训练后获得。

4.1.3 基线方法及其实验参数设置

除第三章引言中已经介绍过的 E2E [2] 和 TPSR [4] 外, 本章实验还选取以下三种算法作为基线方法进行比较:

(1) SymbolicGPT

SymbolicGPT [7] 是一种直接生成式的端到端符号回归方法。该方法利用生成式 Trans-

¹<https://github.com/EpistasisLab/pmlb/tree/master/datasets>

former 根据输入数值样本直接生成表达式。其特点是将表达式拆分为更细粒度的符号单元进行自回归生成，即把一个完整表达式进一步表示为按顺序排列的基本符号序列，再逐步预测后续符号；同时将具体常数替换为占位符，因此模型先生成只包含运算符、变量和常数位置的表达式骨架，再通过后续优化恢复常数。

(2) NeSymReS

NeSymReS^[6] (Neural Symbolic Regression that Scales) 是一种直接生成式的端到端符号回归方法。该方法利用 **Transformer** 根据输入数值样本直接生成表达式序列，并在生成阶段将数值常数统一替换为占位符，因此先预测表达式骨架，再通过数值优化确定常数取值。这里的表达式骨架，是指仅保留运算符、变量和常数所在位置，而不直接给出常数具体数值的表达式形式。

(3) SR4MDL

SR4MDL^[5] (Symbolic Regression via MDLformer-guided Search) 是一种搜索增强式的符号回归方法。该方法利用神经网络 **MDLformer** 估计候选表达式的最小描述长度，并以此引导搜索过程对候选表达式进行扩展和筛选。与直接生成式方法一次性输出表达式不同，SR4MDL 仍依赖搜索逐步寻找目标表达式；与 **TPSR** 主要根据拟合误差引导搜索不同，SR4MDL 将搜索目标转向最小描述长度。

实验中采用训练阶段选出的最优权重进行测试，其主体模型为 **EditFlow** 编码器-解码器 **Transformer**：数值嵌入器维度设为 64，编码器与解码器的隐藏维度均为 512，注意力头数均为 16，且二者均堆叠 8 层。模型总参数量为 87,211,854，约为 87.2M。推理阶段采用并行编辑采样与候选筛选策略，采样步数设为 $n_{\text{steps}} = 100$ ，候选搜索宽度设为 30，并在生成结束后使用 **BFGS** 对表达式中的常数项进行进一步优化。

对比实验中，**E2E** 采用作者公开实现中的预训练权重¹进行测试。其模型主体为 **seq2seq Transformer**，**embedding** 维度为 512，注意力头数为 16，总参数量约 86M，编码器和解码器层数分别为 4 和 16；推理阶段采用 **bagging** 和候选筛选策略，主实验设置为 $B = 100$ 、 $C = 10$ 、 $K = 10$ ，并使用 **BFGS** 对预测常数进行进一步优化。

NeSymReS 使用作者提供的预训练权重²进行测试。其模型由 **Set Transformer** 编码器和 **Transformer** 解码器组成，其中编码器包含 5 个 **ISAB** 模块和 1 个 **PMA** 聚合模块，隐藏维度为 512，注意力头数为 8，诱导点数量设为 $m = 50$ ；解码器为 5 层 **Transformer decoder**，隐藏维度为 512。推理阶段采用 **beam search** 生成候选表达式骨架，**beam size** 设为 32，并通过 **BFGS** 对常数进行拟合，每个候选表达式进行 4 次 **BFGS** 重启。

¹<https://github.com/facebookresearch/symbolicregression>

²<https://huggingface.co/TommasoBendinelli/NeuralSymbolicRegressionThatScales/tree/main>

SymbolicGPT 使用作者公开实现中的预训练权重¹ 进行实验。其主要配置为：嵌入维度 $e = 512$ ，GPT 部分包含 8 个 Transformer block，生成时采用 top- k 采样， $k = 40$ ；训练 4 个 epoch，batch size 为 64，最大输出长度限制为 200 token。

TPSR 使用作者公开代码实现进行实验，并以 E2E 的预训练模型作为骨干网络。其主要配置为：默认树搜索宽度 $k_{\max} = 3$ ，rollout 次数 $r = 3$ ，simulation beam size $b = 1$ ；奖励函数中的复杂度权重通常取 $\lambda = 0.1$ 。对于超过 200 个点的数据集，采用 bagging 方式处理，最多使用 $B = 10$ 个 bag，每个 bag 含 $N = 200$ 个点；与之对比的 E2E 基线也统一采用 $B = 10$ 、候选数 $C = 10$ 、精炼数 $K = 10$ 的设置。TPSR 在扩展阶段使用 top- k 候选，在评估阶段结合 beam search 完成序列，并通过 BFGS 优化常数。

SR4MDL 使用作者公开代码提供的预训练权重² 进行实验。其核心模型 MDLformer 由嵌入器、Transformer 编码器、注意力池化层和 MLP 读出头组成：输入先被嵌入到 $D_i = 64$ 维空间，再经带一层隐藏层的 MLP 映射到 $D_f = 512$ 维；编码器采用 8 层 Transformer encoder，注意力头数为 8，隐藏维度为 512，前馈网络维度为 2048，并限制输入点数不超过 $N_{\max} = 200$ 。推理阶段采用基于 MCTS 的搜索，默认搜索宽度 $k_{\max} = 3$ 、rollout 次数 $r = 3$ 、simulation beam size $b = 1$ ，复杂度权重通常取 $\lambda = 0.1$ 。

4.2 实验结果分析

本节将 EditFlowSR 与多种基线方法进行比较，包括直接生成式端到端符号回归方法和搜索增强式符号回归方法，从拟合精度、表达式复杂度和推理效率三个维度进行系统评估。

4.2.1 与直接生成式方法的比较

本小节将 EditFlowSR 与直接生成式端到端符号回归方法 E2E^[2]、SymbolicGPT^[7] 和 NeSymReS^[6] 进行比较。图 4-1 展示了上述方法在 Feynman、Strogatz 和 Black-box 三类数据集上、不同噪声水平下的 R^2 、表达式复杂度和推理时间对比结果。每组实验重复 20 次³，图中折线为各指标的中位数，阴影区域为各指标的 [0.25, 0.75] 分位数区间。

在 R^2 这一指标上，EditFlowSR 在 Strogatz 和 Black-box 两类数据集上的中位数整体高于直接生成式基线。以 Strogatz 为例，EditFlowSR 在各噪声水平下的 R^2 基本都保

¹<https://github.com/mojivalipour/symbolicgpt>

²<https://github.com/tsinghua-fib-lab/SR4MDL>

³按表 4-1 中的任务数统计，三类数据集共包含 $119 + 14 + 57 = 190$ 个任务。图 4-1 共统计 4 个噪声水平；按本文主实验设置，EditFlowSR 采用候选搜索宽度 30，因此累计候选解码数为 $20 \times 30 \times 190 \times 4 = 456,000$ 。从最终结果汇总看，各组实验中用于统计的解码输出中合法表达式占比为 71%，说明当前模型已能够较稳定地学习前缀表达式语法，因此本文未额外加入显式合法性约束。

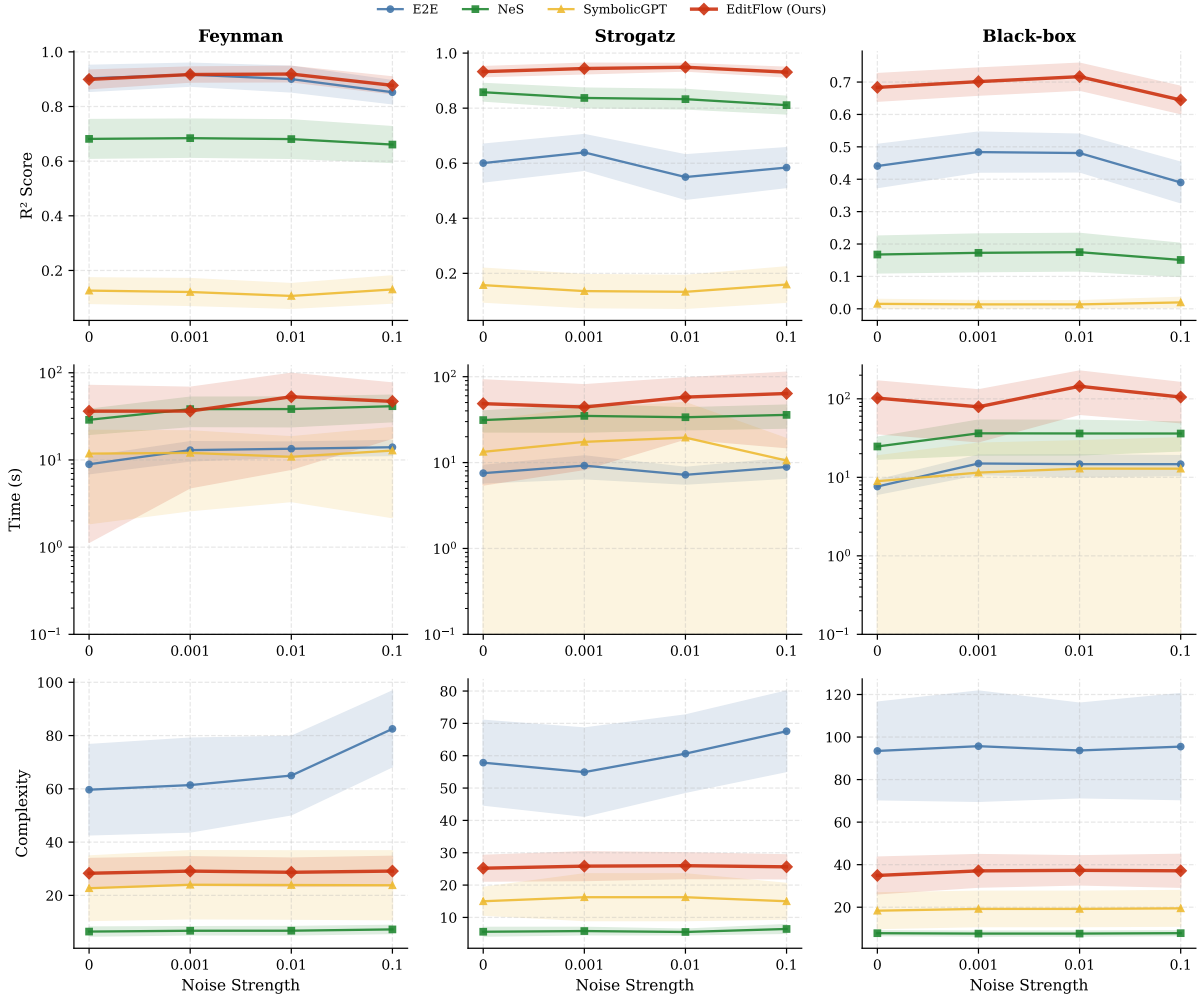


图 4-1 EditFlowSR 与直接生成式方法在三类数据集上的性能对比（折线为中位数，阴影区域为 20 次重复实验的 [0.25, 0.75] 分位数区间）

持在 0.93 以上，而 E2E 在 0.55 ~ 0.64 之间；在 Black-box 数据集上，EditFlowSR 为 0.65 ~ 0.72，E2E 为 0.39 ~ 0.48。在 Feynman 数据集上，EditFlowSR 与 E2E 总体接近，在较高噪声下略高。

在表达式复杂度上，EditFlowSR 的结果整体更低。三类数据集上，EditFlowSR 生成结果的复杂度大致稳定在 25 ~ 37，而 E2E 通常达到 58 ~ 95，并且在噪声增大时还有继续上升的趋势。例如在 Feynman 数据集上，E2E 的复杂度从约 60 上升到 82，而 EditFlowSR 维持在 30 左右。

在推理时间上，EditFlowSR 并不占优，整体耗时高于 E2E、SymbolicGPT 和 NeSym-ReS。结合前两项指标可以看出，EditFlowSR 与直接生成式方法的差异主要体现在更高的 R^2 或更低的表达式复杂度，而不是更短的推理时间。

4.2.2 与搜索增强式方法的比较

本小节将 EditFlowSR 与搜索增强式符号回归方法 TPSR^[4] 和 SR4MDL^[5]（以下简称 MDL）进行比较。图 4-2 展示了上述方法在 Feynman、Strogatz 和 Black-box 三类数据集上、不同噪声水平下的 R^2 、表达式复杂度和推理时间对比结果。与上一组实验相同，每组实验重复 20 次，折线为各指标中位数，阴影区域为各指标的 [0.25, 0.75] 分位数区间。

在 R^2 指标上，EditFlowSR 与搜索增强式方法处于接近或更高水平。以 Black-box 数据集为例，EditFlowSR 在噪声强度为 0.001 和 0.01 时的 R^2 分别为 0.700 和 0.715，高于 MDL 的 0.558 和 0.546，也高于 TPSR 的 0.490 和 0.482；在 Strogatz 数据集上，EditFlowSR 始终保持在 0.93 以上，高于 MDL 在高噪声下的 0.863，并与 TPSR 接近。

在推理时间上，EditFlowSR 低于两种搜索增强方法。例如在 Feynman 数据集上，EditFlowSR 的时间约为 36 ~ 53 s，而 TPSR 为 135 ~ 243 s，MDL 为 344 ~ 441 s；在 Black-box 数据集上，EditFlowSR 约为 80 ~ 140 s，TPSR 约为 300 ~ 354 s，MDL 为 500 ~ 583 s。这一结果与搜索方法需要反复展开、评估和回溯候选表达式的过程一致。

在表达式复杂度上，EditFlowSR 的结果也更低。以 Black-box 数据集为例，EditFlowSR 的复杂度基本稳定在 35 ~ 37，MDL 为 45 ~ 47，TPSR 为 82 ~ 84；在 Feynman 数据集上，EditFlowSR 约为 29，MDL 为 37 ~ 53，TPSR 为 57 ~ 67。结合 R^2 的变化可以看到，TPSR 在无噪声情形下虽然在部分数据集上取得更高 R^2 ，例如 Feynman 上达到 0.971，但在 Black-box 上随噪声加入从 0.713 下降到 0.430，同时复杂度保持在较高水平。整体来看，搜索增强方法在部分场景下可以通过更长的表达式获得较高精度，但对应的复杂度和时间成本也更高。

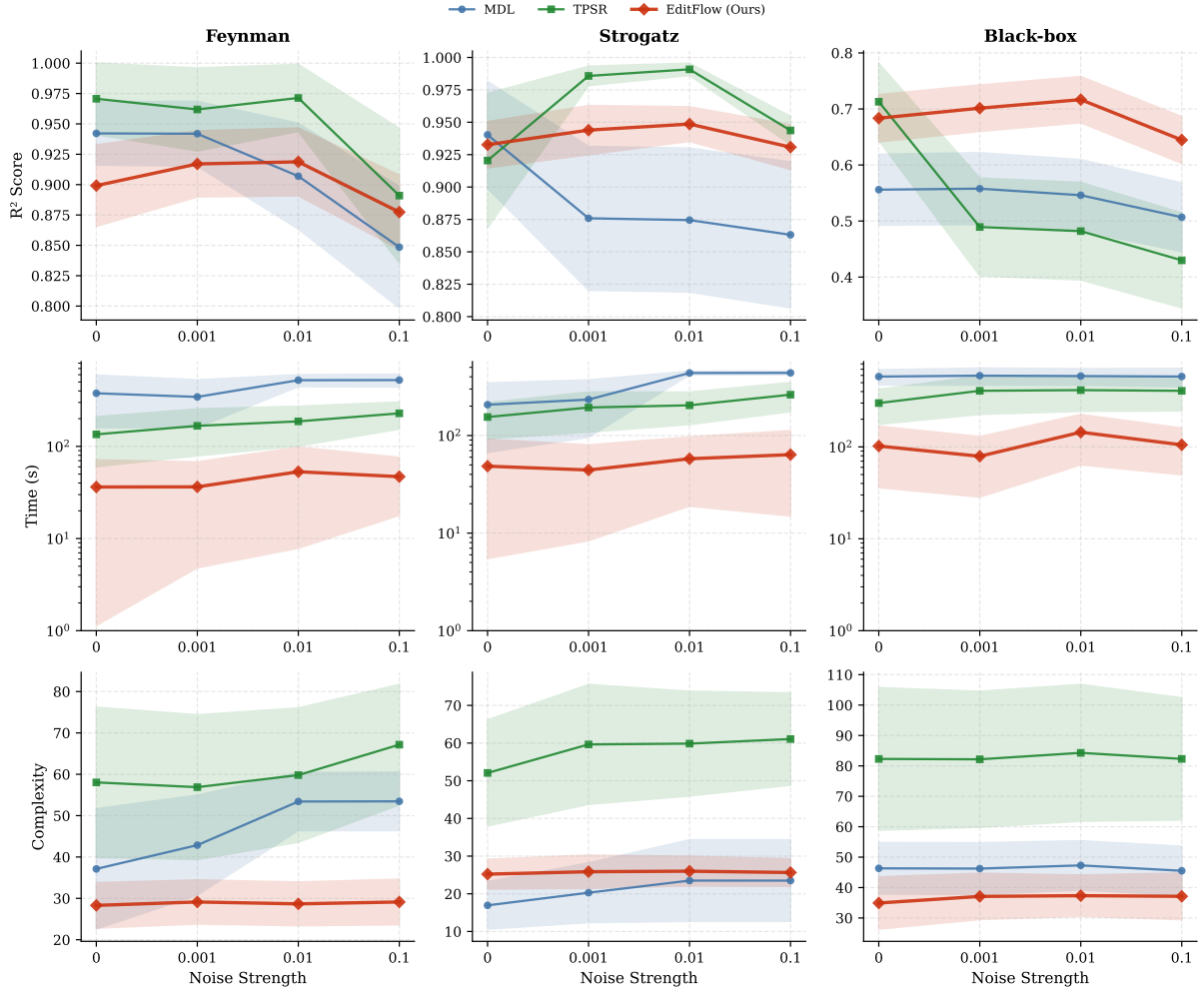


图 4-2 EditFlowSR 与搜索增强式方法在三类数据集上的性能对比 (折线为中位数, 阴影区域为 20 次重复实验的 [0.25, 0.75] 分位数区间)

4.2.3 综合 Pareto 折中分析

为进一步从整体角度比较各方法在拟合精度、表达式复杂度和推理时间之间的折中关系，图 4-3 给出了在噪声强度为 0.1 时，六种方法在全部实验组上的综合 Pareto 前沿结果。图中将三项指标分别转化为相对排名，以避免不同量纲直接比较带来的偏差，并更直观地展示各方法在多目标评价下的整体位置。

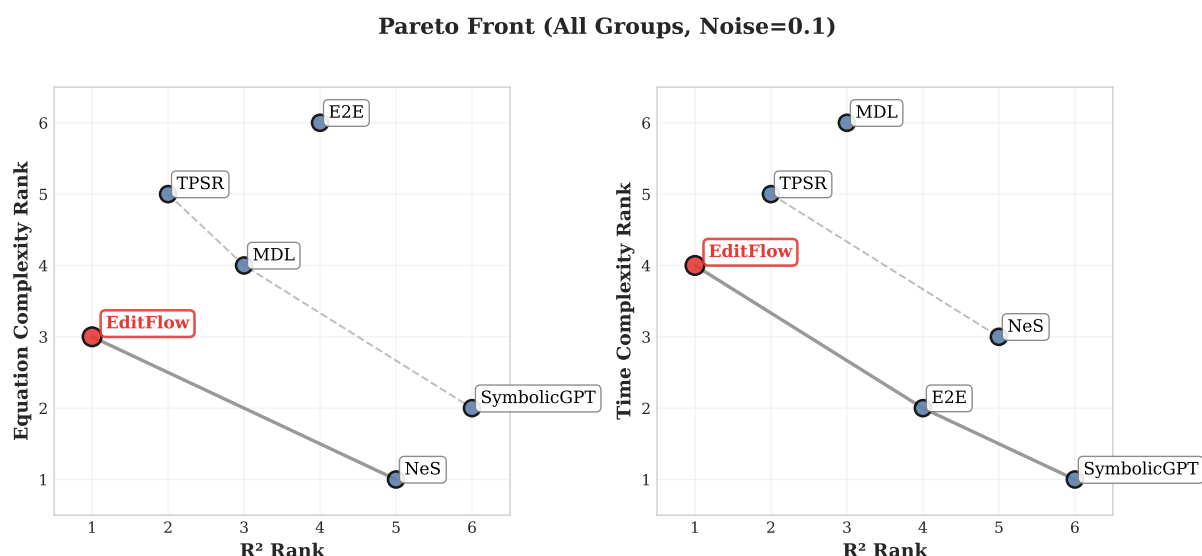


图 4-3 噪声强度为 0.1 时六种方法在全部实验组上的综合 Pareto 折中分析

从图中可以看出，不同方法在三项指标之间存在折中关系。直接生成式方法通常时间开销较低，但在拟合精度或表达式复杂度上存在短板；搜索增强式方法在部分任务上能够取得较高精度，但往往伴随更大的推理时间和更复杂的表达式。EditFlowSR 在三项指标上的相对排名更接近平衡状态，表明其结果并未明显依赖单一指标的改善来换取其余指标。

4.2.4 编辑过程动态分析

为揭示 EditFlowSR 的编辑行为及其与符号回归任务的关系，本节选取 5 个代表性任务，记录模型推理过程中每一步的候选表达式，并从编辑类型分布、表达式长度演化和典型轨迹三个方面进行分析，以说明编辑式生成如何利用符号表达式的结构特性，将随机 token 序列逐步构建为结构化数学表达式。表 4-2 总结了各任务的基本编辑统计信息。

编辑操作类型的分布

在 5 条轨迹共 488 个真实编辑步中，无变化占比最高 (60.0%)，说明模型在离散推理过程中存在较多保持当前候选表达式不变的步，整体编辑是稀疏发生的。在真正发生

表 4-2 编辑过程统计概览

数据集	总编辑步数	最终 R^2	最终复杂度	初始 → 最终长度	推理时间 (s)
230_machine_cpu	101	0.94	30	20 → 46	43.7
hubble	100	0.89	30	41 → 50	57.3
ideal_gas	98	0.92	30	31 → 50	20.8
leavitt	94	0.98	30	47 → 50	21.1
vdp1	100	0.92	31	46 → 49	71.4
平均	99	0.93	30.2	—	42.9

表 4-3 编辑操作类型分布统计

编辑类型	发生次数	占比
替换 (Replace)	126	25.8%
无变化 (No Change)	293	60.0%
插入 (Insert)	60	12.3%
删除 (Delete)	9	1.8%

更新的步中，替换操作最多 (25.8%)，表明模型更倾向于在已有表达式骨架上进行局部修正；插入操作占 12.3%，删除操作仅占 1.8%。

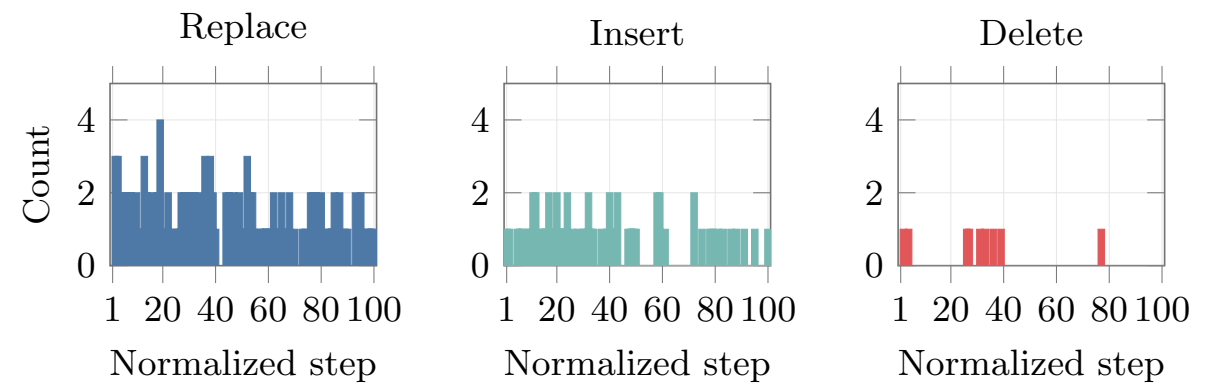


图 4-4 三类编辑操作在归一化 100 步编辑过程中的分布

图 4-4 进一步给出了三类实际编辑操作在归一化 100 步上的柱状分布，其中横轴表示归一化后的编辑位置，纵轴表示对应位置上该操作出现的次数。可以看到，替换操作几乎贯穿整个编辑过程，在前中后段均有分布，说明模型会持续对已有 token 做细粒度修正。插入操作更多集中在前半段，尤其在早期更为密集，这与表达式骨架逐步扩展的过程一致。删除操作的总量最少，且只在少数位置零散出现，说明模型在大多数情况下不依赖频繁删减结构，而是主要通过替换和少量插入完成表达式优化。

表达式结构对比示例

为直观展示上述差异，本节以 Leavitt 数据集为例，分别给出 TPSR 和 EditFlowSR

在该任务上的输出表达式。

TPSR 生成的表达式 ($R^2 = 0.75$, 复杂度 = 42) 为:

$$\begin{aligned}\hat{f}_{\text{TPSR}}(x_0) = & (-13.09 + (-1.68 + 1.91 x_0)) \\ & (4.10 - 0.21 \sin(-1.38 + 90.0(-1.68 + 1.91 x_0))) \\ & (-0.70 - 0.11 \cos(-0.60 + 0.66 \sin(1.05 + \cdots)))\end{aligned}$$

其中省略号表示因复杂度限制而截断的后续结构。

EditFlowSR 生成的表达式 ($R^2 = 0.98$, 复杂度 = 30) 为:

$$\hat{f}_{\text{EFSR}}(x_0) = -5.53 + \frac{15.13}{-26.17(-0.47 + 0.19 x_0)^2 - 0.68 x_0} - 0.11 x_0 + 2.30 \sqrt{99.90 + 15.0 x_0}$$

对比两者可以发现:(1) TPSR 的表达式包含深层嵌套的三角函数组合 $\sin(\cdots \cos(\cdots \sin(\cdots)))$, 这种结构在物理上难以解释; (2) EditFlowSR 的表达式由求逆、平方和开方等基本运算构成, 结构层次更浅、数学含义更清晰。这一对比表明, 序列式生成方法在遇到困难任务时倾向于通过不断嵌套更复杂的函数 (如三角函数的反复嵌套) 来降低残差, 但由于这种嵌套结构是从左到右逐 token 生成的, 模型无法回头删除已经生成的冗余子结构, 最终导致表达式过长且难以解释。相比之下, 编辑式生成在每一步都可以对已有 token 进行插入、替换或删除, 因此能够在迭代过程中主动移除冗余结构, 从而以更少的 token 数量达到更高的拟合精度。

编辑轨迹示例

为直观展示 EditFlowSR 的迭代编辑过程, 本节以 Leavitt 数据集为例, 追踪模型从初始随机 token 序列到最终表达式的完整演化路径。表 4-4 给出了若干关键里程碑处的 token 序列前缀, 其中加粗标记的 token 表示相较于上一步发生替换或插入的位置。

表 4-4 Leavitt 数据集上 token 序列的编辑演化

步骤	token 序列前缀
0	N7236 N6987 E54 N7278 N8567 N4376 N2952 N7187 N5603 N7158 N3323 ... (47 个数值 token)
5	add N7236 N6987 E54 N7278 N8567 N4376 add N7187 N5603 N7158 N3323 ... (4 个运算符)
30	add - E-3 add sub mul E54 N7278 N8567 add mul N5603 N2800 N3323 pow2 ... (12 个运算符, x_0 出现)
50	add - N4110 E-3 add sub mul E54 N7278 E-3 inv add mul N5603 N2800 E-2 pow2 ... (inv 引入)
93	add - N4110 E-3 add sub mul + N5070 E-3 inv add mul - N2800 ... (可解析, $R^2=0.98$)

从该轨迹可以观察到编辑过程的三个典型阶段：

阶段一 (Step 0~5)：运算符出现。初始序列全部由随机数值 token (如 $N7236$, $E54$) 组成。模型在前几步通过替换操作将部分数值 token 修正为运算符，例如将 $N2952$ 替换为 add 、将 $N7158$ 替换为 mul ，同时在序列中插入 $+$ 、 $-$ 等符号，开始构建基本的运算结构。

阶段二 (Step 5~50)：结构塑造。模型通过替换操作引入更多运算符和变量。在 Step 30 前后， sub (减法)、 mul (乘法)、 $pow2$ (平方) 和 $sqrt$ (开方) 运算符出现，首个变量 x_0 被引入，表达式开始具备可解释的数学结构。在 Step 50 前后， inv (求逆) 运算符被引入，常数 token $E54$ 被替换为 $E - 3$ ，表达式骨架基本确立。

阶段三 (Step 50~93)：常数收敛。表达式结构在此阶段趋于稳定，模型主要通过替换操作对常数 token 进行微调。例如， $N7278$ 经过 Step 30 时被替换为 $E - 3$ ，Step 50 时收敛为 $N4110$ (即 -4.11)；类似地， $N5603$ 被替换为 $N5070$ (即 5.07)。最终，模型在 Step 93 生成可完整解析的表达式。

4.3 本章小结

本章在三个数据集上对 EditFlowSR 进行了实验，并与多种符号回归方法进行了比较。实验结果表明，EditFlowSR 具有较好的综合性能，最后对其编辑动态过程和局限性进行了分析。

结论

论文工作总结

本文围绕端到端符号回归中序列式生成范式在表达式结构修正能力上的不足，开展了基于编辑流的符号回归研究。首先，本文从表达式的数学结构特性出发，分析了前缀树结构中运算符、变量与常数之间的层级依赖关系如何导致序列式生成产生结构性误差累积和表达式冗余，明确了现有方法中结构修正能力不足的深层原因。在此基础上，本文提出了一种新的技术思路——将表达式发现过程建模为对候选表达式的多轮迭代编辑，而非固定顺序的单次生成，并设计了基于编辑流的端到端符号回归方法 **EditFlowSR**。该方法通过插入、替换和删除三类 **token** 级编辑操作对当前候选表达式进行全局位置的并行修正，使模型在每一轮迭代中能够同时调整表达式的任意位置，从而突破了序列式生成中早期决策对后续结构的强约束。

在方法实现上，本文结合已有数值编码方法与编辑流生成框架，构建了适用于符号表达式的编辑建模方案，包括条件化编辑流解码、辅助对齐空间标签构造和基于连续时间马尔可夫链的并行编辑推理，使模型能够在数值条件约束下逐步生成具有可解释性的数学表达式。

在实验部分，本文在三个基准数据集上对 **EditFlowSR** 进行了系统评估，这三个数据集分别包含 119 个费曼方程、14 个 **ODEStrogatz** 挑战和 57 个没有已知底层函数的黑盒回归任务。实验结果表明，在与 **E2E**、**SymbolicGPT** 和 **NeSymReS** 等直接生成式方法的比较中，**EditFlowSR** 在多数设置下取得了更高的 R^2 且表达式复杂度更低；在与 **TPSR** 和 **SR4MDL** 等搜索增强式方法的比较中，**EditFlowSR** 的 R^2 处于接近或更高水平，同时推理时间更短，在拟合精度、表达式简洁性与推理效率三者之间展现出较好的平衡。综合 **Pareto** 折中分析进一步验证了这一观察。此外，对编辑过程动态行为的分析揭示了表达式发现的三个典型阶段——运算符出现、结构塑造和常数收敛——这一由粗到细的优化模式与数学表达式的层级结构高度契合，为理解编辑式生成在符号回归中的有效性提供了直观的实证支撑。同时，对失败案例的分析揭示了模型在高维输入、常数密集和等价形式歧义等场景中的局限性。

工作展望

尽管本文提出的 **EditFlowSR** 在符号回归任务中取得了较好的实验结果，但仍有进一步改进和拓展的空间。

1) 本文方法目前主要在包含 119 个费曼方程、14 个 ODEStrogatz 挑战和 57 个没有已知底层函数的黑盒回归任务的基准数据集上进行了评估, 未涉及更高维或真实科学发现场景的数据集。对失败案例的分析表明, 模型在高维输入、常数密集和等价形式歧义等场景中存在局限。后续可进一步在更具挑战性的数据集上检验方法的适用性, 并针对高维输入增强数值条件编码能力。

2) 当前方法虽然在表达式复杂度和推理效率之间取得了较好平衡, 但在部分任务上与搜索增强方法相比仍存在一定精度差距。未来可以考虑将编辑式生成与轻量级搜索机制结合, 在不过多增加推理开销的前提下进一步提升表达式恢复能力。

3) 本文未对 EditFlowSR 的各个设计组件进行消融实验, 因此无法量化各组件(如并行编辑策略、辅助对齐空间、CTMC 推理框架等)对最终性能的独立贡献。后续工作可通过控制变量实验明确各模块的作用, 进一步强化方法设计的科学验证。

4) 本文对编辑过程动态行为的分析主要基于定性观察, 尚未建立编辑阶段与最终表达式质量之间的定量关联。后续可通过引入自动阶段检测和阶段性评估指标, 将编辑动力学的分析从描述性观察提升为可验证的实验结论。

参考文献

- [1] LA CAVA W, ORZECOWSKI P, BURLACU B, et al. Contemporary symbolic regression methods and their relative performance[A]. 2021.
- [2] KAMIENNY P A, D'ASCOLI S, LAMPLE G, et al. End-to-end symbolic regression with transformers[J]. Advances in Neural Information Processing Systems, 2022, 35: 10269-10281.
- [3] VASTL M, KULHÁNEK J, KUBALÍK J, et al. Symformer: end-to-end symbolic regression using transformer-based architecture[J]. IEEE Access, 2024, 12: 37840-37849.
- [4] SHOJAEE P, MEIDANI K, BARATI FARIMANI A, et al. Transformer-based planning for symbolic regression[J]. Advances in Neural Information Processing Systems, 2023, 36: 45907-45919.
- [5] YU Z, DING J, LI Y, et al. Symbolic regression via MDLformer-guided search: from minimizing prediction error to minimizing description length[A]. 2024.
- [6] BIGGIO L, BENDINELLI T, NEITZ A, et al. Neural symbolic regression that scales[C]// International Conference on Machine Learning. PMLR, 2021: 936-945.
- [7] VALIPOUR M, YOU B, PANJU M, et al. SymbolicGPT: a generative transformer model for symbolic regression[A]. 2021.
- [8] HAVASI M, KARRER B, GAT I, et al. Edit flows: flow matching with edit operations[A]. 2025: arXiv:2506.09018.
- [9] LI Q, HU Y, LIU J, et al. GENSR: symbolic regression based in equation generative space [A]. 2026.
- [10] UDRESCU S M, TEGMARK M. AI feynman: a physics-inspired method for symbolic regression[J/OL]. Science Advances, 2020, 6(16): eaay2631. DOI: 10.1126/sciadv.aay2631.
- [11] OLSON R S, LA CAVA W, ORZECOWSKI P, et al. PMLB: a large benchmark suite for machine learning evaluation and comparison[J]. BioData Mining, 2017, 10: 1-13.

致谢

四年时间转眼即逝，青涩而美好的本科生活快告一段落了。回首这段时间，我不仅学习到了很多知识和技能，而且提高了分析和解决问题的能力与养成了一定的科学素养。虽然走过了一些弯路，但更加坚定我后来选择学术研究的道路，实在是获益良多。这一切与老师的教诲和同学们的帮助是分不开的，在此对他们表达诚挚的谢意。

首先要感谢的是我的指导老师王大明教授。我作为一名本科生，缺少学术研究经验，不能很好地弄清所研究问题的重点、难点和热点，也很难分析自己的工作所能够达到的层次。王老师对整个研究领域有很好的理解，以其渊博的知识和敏锐的洞察力给了我非常有帮助的方向性指导。他严谨的治学态度与辛勤的工作方式也是我学习的榜样，在此向王老师致以崇高的敬意和衷心的感谢。

最后我要感谢我的家人，正是他们的无私的奉献和支持，我才有了不断拼搏的信心和勇气，才能取得现在的成果。

王小明

2026年3月30日

附录 A 补充更多细节

A.1 补充图

A.1.1 补充图

这是附录内容，应该用宋体小四号字体。

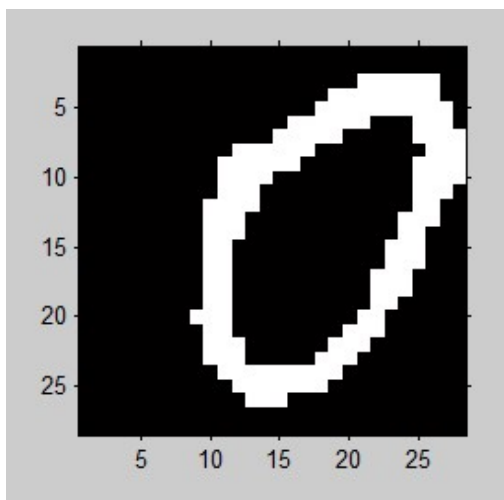


图 A-1 实验训练集

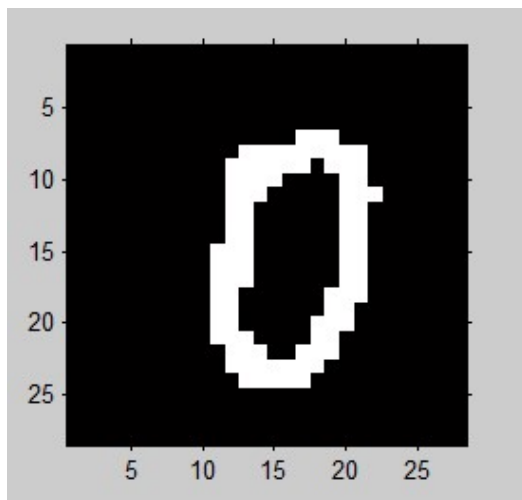


图 A-2 实验测试集